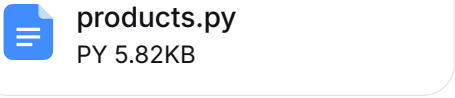
hello i want you to explain the folloging code word by word ,line by line for a beginawhat teach code does and etc

I'd be happy to explain code line by line! However, I notice you said "the following code" but I don't see any code in your message.

Could you please paste the code you'd like me to explain? Once you share it, I'll break it down:

- **Word by word** — explaining what each keyword, variable, and symbol means
- **Line by line** — what each line does and why it's there
- **Overall purpose** — what the program is trying to accomplish
- **Beginner-friendly** — using simple language and analogies where helpful

Go ahead and paste the code whenever you're ready!



Full Walkthrough of `products.py`

This file defines a set of **API endpoints** for managing "products" in an online store. It uses:

- **FastAPI** → a Python web framework for building APIs
- **Supabase** → a hosted database + file storage service (like Firebase, but using Postgres)

Think of this file as the **"Products" section of a store's backend**. Customers can *list* and *view* products; only admins can *create*, *update*, or *delete* them.

Let me break it down section by section.

1. Imports

```python
from fastapi import APIRouter, UploadFile, File, Form, HTTPException, Depends, Query
from app.database import supabase
from app.dependencies import require_admin
import uuid
from typing import List, Optional
```

Import	What it is / does
`APIRouter`	A mini-app used to group related routes (like `/products`). Keeps code modular.
`UploadFile, File`	Used when a client uploads **files** (e.g., product images).
`Form`	Tells FastAPI the incoming value comes from a **form field** (not JSON).
`HTTPException`	Lets you return an error response like `404 Not Found`.
`Depends`	FastAPI's dependency injection — runs a function (like an auth check) before the endpoint runs.
`Query`	Declares a value comes from the **URL query string** (`?search=phone`).
`supabase`	Your database + storage client, imported from `app.database`.
`require_admin`	A custom function that verifies the caller is an admin.
`uuid`	Generates **unique IDs** (used for filenames).
`List, Optional`	Type hints: `List[X]` = a list of X, `Optional[X]` = X or None.

2. Router Setup

2. Router Setup

```python
router = APIRouter(prefix="/products", tags=["products"])
```

- `APIRouter` → creates a group of routes.
- `prefix="/products"` → every route here is automatically prefixed with `/products`. So `@router.get("/")` becomes `GET /products/`.
- `tags=["products"]` → just a label used in auto-generated docs (Swagger UI).

3. Commented-Out Old Version

```python
# @router.get("/")
# async def get_products():
#     response = supabase.table("products").select("*").execute()
#     return {"data": response.data}
```

This is an older, simpler version of `get_products`. It was replaced by the fancier version below that supports search, filters, and sorting. Commented out so it doesn't run — kept as reference.

4. GET `/products/` — List / Search Products

```python
@router.get("/")
def get_products(
    search: Optional[str] = Query(None, description="Search by name"),
    category: Optional[str] = Query(None, description="Filter by category"),
    brand: Optional[str] = Query(None, description="Filter by brand"),
    color: Optional[str] = Query(None, description="Filter by color"),
    min_price: Optional[float] = Query(None, description="Minimum price"),
    max_price: Optional[float] = Query(None, description="Maximum price"),
```

```
    sort_by: Optional[str] = Query("created_at", description="Sort field: price or crea
ted_at"),
    order: Optional[str] = Query("desc", description="Sort order: asc or desc")
):
```

This declares a **GET endpoint**. The function parameters are all **query parameters** — the client passes them in the URL, e.g.:

```
text

GET /products/?search=iphone&brand=Apple&min_price=500
```

- Each is `Optional[...] = Query(None)` → if not provided, defaults to `None`.
- `sort_by` defaults to `"created_at"`, `order` defaults to `"desc"`.
- The `description=` strings appear in the auto-generated API docs.

The body

```
python

    try:
        query = supabase.table("products").select("*")
```

- `supabase.table("products")` → points to the `products` table.
- `.select("*")` → "I want all columns."
- **Important:** this doesn't hit the DB yet — it just *builds* a query object.

```
python

        if search:
            query = query.ilike("name", f"%{search}%")
```

- `if search:` → only run if a search term was given.
- `.ilike(...)` → case-insensitive "LIKE" match.
- `f"%{search}%"` → pattern with wildcards: `%iphone%` matches any name containing "iphone".

- Reassigning `query` builds up a chain of conditions.

python

```
        if category:
            query = query.eq("category", category)
        if brand:
            query = query.eq("brand", brand)
        if color:
            query = query.eq("color", color)
```

- `.eq(...)` → "equals". Adds filters for exact matches.

python

```
        if min_price is not None:
            query = query.gte("price", min_price)
        if max_price is not None:
            query = query.lte("price", max_price)
```

- `.gte(...)` → greater-than-or-equal.
- `.lte(...)` → less-than-or-equal.
- We use `is not None` (not just `if min_price`) because `0.0` is a valid price but is falsy in Python.

python

```
        sort_field = sort_by or "created_at"
        order_value = (order or "desc").lower()
        is_desc = order_value == "desc"
        query = query.order(sort_field, desc=is_desc)
```

- `x or y` → use `x` if it's truthy, else `y` (defensive fallback).
- `.lower()` → normalize `"DESC"` → `"desc"`.
- `is_desc` → boolean: are we sorting descending?
- `.order(...)` → applies sorting.

python

```
        response = query.execute()
        return {"success": True, "data": response.data}
```

- `.execute()` → **this is when the query actually runs**.
- `response.data` → the list of rows Supabase returns.
- Returns a clean JSON object.

python

```
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

- If anything blows up (bad query, DB down, etc.), return a `500 Internal Server Error` with the error message.

5. POST `/products/` — Create a Product (admin only)

python

```
@router.post("/")
async def create_product(
    name: str = Form(...),
    price: float = Form(...),
    brand: str = Form(...),
    images: List[UploadFile] = File(...),
    description: str = Form(...),
    color: str = Form(...),
    admin_user = Depends(require_admin)
):
```

- `@router.post("/")` → handles `POST /products/`.
- `async def` → this endpoint does I/O (uploads, DB), so it's asynchronous.
- Each `Form(...)` → required field in a **multipart form**.
- `images: List[UploadFile] = File(...)` → **one or more** uploaded files.
- `admin_user = Depends(require_admin)` → runs `require_admin` first; if it raises, the

endpoint never executes. If it returns, its value goes into `admin_user`.

Step 1 — Create the product record

```python
    new_product = {"name": name, "price": price, "brand": brand, "description": descrip
tion, "color": color}
    product_response = supabase.table("products").insert(new_product).execute()
```

- Build a dict of the product data.
- `.insert(...)` → adds a row. `.execute()` → runs it.
- `product_response.data` will contain the newly inserted row(s).

```python
    product_data = product_response.data
    if not isinstance(product_data
```



profiles.py
PY 2.38KB

Line-by-Line Explanation of `profiles.py`

This file defines API routes for managing **user profiles** (like name, phone, address, avatar, etc.). Let's go through it piece by piece.

1. Imports

```python
from fastapi import APIRouter, HTTPException, Depends
```

- `fastapi` — a modern Python web framework used to build APIs.
- `APIRouter` — a helper that lets you group related routes together (like a mini-app for profiles)

profiles).

- `HTTPException` — used to return error responses (e.g., 404 Not Found).
- `Depends` — tells FastAPI to run another function (a "dependency") before running the route (used here for authentication).

```
python

from pydantic import BaseModel
```

- `pydantic` — a library for validating data.
- `BaseModel` — the base class you inherit from to define a data "shape" (schema). FastAPI uses it to validate incoming JSON automatically.

```
python

from typing import Optional
```

- `Optional[X]` — means "this field can either be of type X or be `None` (missing)".

```
python

from datetime import date
```

- `date` — Python's built-in type for calendar dates (YYYY-MM-DD).

```
python

from app.database import supabase
```

- `supabase` — a client object connected to your Supabase database (Postgres + storage + auth). You use it to run queries like `.table(...).select(...)`.

```
python

from app.dependencies import get_current_user
```

- `get_current_user` — a custom function that verifies the user's login token and returns the logged-in user. It's used with `Depends()`.

2. Create the Router

```python
router = APIRouter(prefix="/profiles", tags=["Profiles"])
```

- Creates a router object.
- `prefix="/profiles"` — every route defined here will start with `/profiles` (e.g., `/profiles/me`).
- `tags=["Profiles"]` — groups these routes under "Profiles" in the auto-generated API docs (Swagger UI at `/docs`).

3. Define the Data Shape (Schema)

```python
class ProfileUpdate(BaseModel):
    full_name: Optional[str] = None
    phone_number: Optional[str] = None
    address: Optional[str] = None
    avatar_url: Optional[str] = None
    banner_url: Optional[str] = None
    date_of_birth: Optional[date] = None
```

- Defines what a profile update request looks like.
- Every field is **optional** with a default of `None`, so the user can send just the fields they want to change.
- FastAPI automatically:
  - Validates incoming JSON against this shape.
  - Converts `date_of_birth` strings (like `"1995-05-10"`) into real `date` objects.
  - Rejects invalid data with a 422 error.

4. GET My Profile

```python
@router.get("/me")
def get_my_profile(current_user = Depends(get_current_user)):
```

- `@router.get("/me")` — a decorator that registers this function to handle `GET /profiles/me`.
- `current_user = Depends(get_current_user)` — before this function runs, FastAPI calls `get_current_user()`. If the token is invalid, it throws an error. If valid, it passes the user object into `current_user`.

```python
    try:
        res = supabase.table("profiles").select("*").eq("user_id", current_user.id).sin
gle().execute()
        return {"success": True, "data": res.data}
```

- `supabase.table("profiles")` — pick the `profiles` table.
- `.select("*")` — fetch all columns.
- `.eq("user_id", current_user.id)` — filter rows where `user_id` equals the logged-in user's ID.
- `.single()` — expect exactly one row (throws an error otherwise).
- `.execute()` — actually run the query.
- Returns a JSON object with a success flag and the profile data.

```python
    except Exception:
        raise HTTPException(status_code=404, detail="Profile not found")
```

- If anything went wrong (no profile found, DB error), return a **404** with a message.

5. POST (Create/Overwrite) My Profile

```
python

@router.post("/me")
def upsert_my_profile(payload: ProfileUpdate, current_user = Depends(get_current_use
r)):
```

- Handles `POST /profiles/me`.
- `payload: ProfileUpdate` — FastAPI parses the request body JSON into a `ProfileUpdate` object automatically.
- Still requires an authenticated user.

```
python

    try:
        data = payload.dict(exclude_unset=True)
```

- `.dict()` — converts the Pydantic object into a plain Python dictionary.
- `exclude_unset=True` — only include fields the user actually sent (don't include defaults as `None`).

```
python

        if "date_of_birth" in data and isinstance(data["date_of_birth"], date):
            data["date_of_birth"] = data["date_of_birth"].isoformat()
```

- If `date_of_birth` was provided and it's a `date` object, convert it to a string like `"1995-05-10"` so the JSON/database layer can handle it.

```
python

        data["user_id"] = current_user.id
```

- **Important security step**: forcibly set `user_id` to the authenticated user's ID. This prevents someone from spoofing another user's profile.

```python
        res = supabase.table("profiles").upsert(data, on_conflict="user_id").execute()
        return {"success": True, "data": res.data}
```

- `.upsert(...)` — "update or insert". If a row with that `user_id` exists, update it; otherwise insert a new one.
- `on_conflict="user_id"` — says the `user_id` column is the unique key that determines conflict.

```python
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e))
```

- On error, return a **400 Bad Request** with the error message.

6. PATCH (Partial Update) My Profile

```python
@router.patch("/me")
async def update_user_profile(profile_data: ProfileUpdate, current_user = Depends(get_c
urrent_user)):
```

- Handles `PATCH /profiles/me`.
- `async def` — this route is asynchronous (fine, FastAPI supports both).
- `profile_data: ProfileUpdate` — same validation as before.

```python
    data_to_update = profile_data.dict(exclude_unset=True)
```

- Only include fields the user actually sent.
- **Why it matters**: if a field weren't sent, we don't want to overwrite it with `None`.

```python
    if "date_of_birth" in data_to_update and isinstance(data_to_update["date_of_birth"], date):
        data_to_update["date_of_birth"] = data_to_update["date_of_birth"].isoformat()
```

- Same date conversion as before.

```python
    response = supabase.table("profiles").update(data_to_update).eq("user_id", current_user.id).execute()
```

- `.update(...)` — update the given fields.
- `.eq("user_id", current_user.id)` — only update the row that belongs to the logged-in user.

```python
    return {"message": "Profile updated!", "data": response.data}
```

- Return a success message plus the updated row.

7. Big Picture Summary

Route	Method	Purpose	Auth Required?
`/profiles/me`	GET	Fetch my profile	✅ Yes
`/profiles/me`	POST	Create or replace my profile	✅ Yes
`/profiles/me`	PATCH	Partially update my profile	✅ Yes

Key concepts used:

1. **FastAPI dependency injection** (`Depends`) — automatically runs auth before the route.

2. **Pydantic models** — define/validate the shape of incoming JSON.
3. `exclude_unset=True` — crucial for PATCH/POST so you only send what the user provided.
4. `user_id = current_user.id` — always enforce ownership server-side for security.
5. `upsert` — one query that either inserts or updates.
6. `.single()` — expects one row and errors if there are zero or many.

8. Two Small Improvements You Could Make

1. **Bug in POST/PATCH date handling**: `payload.dict()` in Pydantic v2 is deprecated — you'd use `payload.model_dump(exclude_unset=True)`. Also, `isinstance(..., date)` works but if you want to check `datetime` too, use `from datetime import date, datetime`.
2. **Consistency**: GET uses `try/except` for 404 but PATCH doesn't check if the row existed — you could verify `response.data` isn't empty before returning "updated".

Would you like me to also walk through the `products.py` file the same way, or compare how the two files use auth differently?

products.py
PY 5.82KB

Full Line-by-Line Explanation of `products.py`

This file defines the **Products API** for an e-commerce backend. It handles listing, creating, updating, deleting, and viewing single products. It uses **FastAPI** (web framework) and **Supabase** (database + file storage).

Let's go through it section by section.

1. Imports

1. Imports

```python
from fastapi import APIRouter, UploadFile, File, Form, HTTPException, Depends, Query
from app.database import supabase
from app.dependencies import require_admin
import uuid
from typing import List, Optional
```

Import	Purpose
`APIRouter`	A mini-app that groups related routes (e.g., all product routes).
`UploadFile , File`	For handling **file uploads** (product images).
`Form`	Says the value comes from a **multipart form field**, not JSON.
`HTTPException`	Returns HTTP errors (like `404 , 500`).
`Depends`	FastAPI dependency injection — runs a helper function first (like auth).
`Query`	Says a value comes from the **URL query string** (`?search=phone`).
`supabase`	The database + storage client.
`require_admin`	Custom function that checks the caller is an admin.
`uuid`	Generates universally-unique IDs (used in filenames).
`List , Optional`	Type hints: `List[X]` = list of X, `Optional[X]` = X or None.

2. Create the Router

```python
router = APIRouter(prefix="/products", tags=["products"])
```

- Creates a route group.
- `prefix="/products"` → every route here begins with `/products`.
- `tags=["products"]` → label shown in the auto-generated API docs (Swagger UI).

3. Commented-Out Old Version

```python
# @router.get("/")
# async def get_products():
#     response = supabase.table("products").select("*").execute()
#     return {"data": response.data}
```

An older, simpler version of the "list products" endpoint. It was replaced by the richer version below (which supports search, filters, sorting). Kept as reference but not run.

4. GET `/products/` — List / Search / Filter Products

```python
@router.get("/")
def get_products(
    search: Optional[str] = Query(None, description="Search by name"),
    category: Optional[str] = Query(None, description="Filter by category"),
    brand: Optional[str] = Query(None, description="Filter by brand"),
    color: Optional[str] = Query(None, description="Filter by color"),
    min_price: Optional[float] = Query(None, description="Minimum price"),
    max_price: Optional[float] = Query(None, description="Maximum price"),
    sort_by: Optional[str] = Query("created_at", description="Sort field: price or crea
ted_at"),
    order: Optional[str] = Query("desc", description="Sort order: asc or desc")
):
```

- `@router.get("/")` → handles `GET /products/`.
- All parameters are **query parameters** — sent in the URL, e.g.:

```
text

GET /products/?search=iphone&brand=Apple&min_price=500&sort_by=price&order=asc
```

- Each is `Optional[...] = Query(None)` → defaults to `None` if not provided.
- `sort_by` defaults to `"created_at"`, `order` defaults to `"desc"`.
- `description=` strings show up in the API docs.

Body of the function

```
python

    try:
        query = supabase.table("products").select("*")
```

- Points to the `products` table.
- `.select("*")` → "fetch all columns".
- **This does NOT hit the DB yet** — it *builds* a query object. Think of it like writing a SQL statement but not running it.

```
python

        if search:
            query = query.ilike("name", f"%{search}%")
```

- `if search:` → only add this filter if a search term was given.
- `.ilike(...)` → case-insensitive "LIKE" match.
- `f"%{search}%"` → pattern with wildcards. `%iphone%` matches any name containing "iphone".
- Reassign `query` because each method returns a new query object.

```
python
```

```
python
```

```
        if category:
            query = query.eq("category", category)
        if brand:
            query = query.eq("brand", brand)
        if color:
            query = query.eq("color", color)
```

- `.eq(...)` → exact "equals" filter.
- Each only runs if the user provided that value.

```
python
```

```
        if min_price is not None:
            query = query.gte("price", min_price)
        if max_price is not None:
            query = query.lte("price", max_price)
```

- `.gte(...)` → greater-than-or-equal.
- `.lte(...)` → less-than-or-equal.
- We use `is not None` (not just `if min_price`) because `0.0` is a valid price but is falsy in Python. This is a small but important detail.

```
python
```

```
        sort_field = sort_by or "created_at"
        order_value = (order or "desc").lower()
        is_desc = order_value == "desc"
        query = query.order(sort_field, desc=is_desc)
```

- `x or y` → use `x` if it's truthy, else `y` (fallback defaults).
- `.lower()` → normalizes `"DESC"` → `"desc"`.
- `is_desc` → boolean telling us if sorting is descending.
- `.order(...)` → adds sorting to the query.

```
python
```

```
        response = query.execute()
```

```
        return {"success": True, "data": response.data}
```

- `.execute()` → NOW the query actually runs against the DB.
- `response.data` → list of matching rows.
- Returns a nice JSON object.

python

```
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

- Catch any failure (bad query, DB down) and return `500 Internal Server Error`.

5. POST `/products/` — Create a Product (admin only)

python

```
@router.post("/")
async def create_product(
    name: str = Form(...),
    price: float = Form(...),
    brand: str = Form(...),
    images: List[UploadFile] = File(...),
    description: str = Form(...),
    color: str = Form(...),
    admin_user = Depends(require_admin)
):
```

- `@router.post("/")` → handles `POST /products/`.
- `async def` → endpoint does I/O (network, files) so it's asynchronous.
- Each `Form(...)` → required form field. The `...` means "required".
- `images: List[UploadFile] = File(...)` → **one or more uploaded files**.
- `admin_user = Depends(require_admin)` → before the function runs, `require_admin` is called. If it raises, the endpoint never runs. This enforces **admin-only** access.

Step 1 — Create the product record

```python
    new_product = {"name": name, "price": price, "brand": brand, "description": descrip
tion, "color": color}
    product_response = supabase.table("products").insert(new_product).execute()
```

- Build a dictionary of the product data.
- `.insert(...)` → adds a row.
- `.execute()` → runs it. The response contains the new row (usually including its generated `id`).

```python
    product_data = product_response.data
    if not isinstance(product_data, list) or len(product_data) == 0:
        return {"error": "Failed to create product"}
```

- Extract `product_response.data`.
- Defensive checks: is it actually a list, and is it non-empty? If not, return an error. This avoids crashing on unexpected responses.

```python
    created_product = product_data[0]
    if not isinstance(created_product, dict):
        return {"error": "Failed to create product"}

    product_id = created_product.get("id")
    if product_id is None:
        return {"error": "Failed to create product"}
```

- Get the first (and only) row.
- Make sure it's a dict.
- Safely extract the `id` via `.get("id")` (returns None if missing instead of raising).
- If we couldn't get an ID, we can't attach images — bail out.

Step 2 — Loop through each uploaded image

```python
    for image in images:
        file_bytes = await image.read()
```

- Loop over each uploaded file.
- `await image.read()` → read the raw bytes of the image. `await` because reading is asynchronous.

```python
        if image.filename and "." in image.filename:
            file_extension = image.filename.split(".")[-1]
        else:
            file_extension = "jpg"
```

- Safely extract the file extension.
- `"photo.png".split(".")[-1]` → `"png"`.
- If no filename or no dot, default to `"jpg"`.

```python
        unique_filename = f"{uuid.uuid4()}.{file_extension}"
        file_path = f"phones/{unique_filename}"
```

- Create a **unique filename** using a UUID (prevents collisions if two users upload "photo.jpg").
- Put it in a `phones/` folder inside the storage bucket.

```python
        supabase.storage.from_("product-images").upload(file_path, file_bytes)
        image_url = supabase.storage.from_("product-images").get_public_url(
            file_path
        )
```

- Upload the raw bytes to the `product-images` storage bucket.
- Get back a **public URL** so the image can be displayed in the frontend.

Step 3 — Save the image URL in the database

```python
        new_image = {"product_id": product_id, "image_url": image_url}
        supabase.table("product_images").insert(new_image).execute()
```

- Insert a row into the `product_images` table linking the image to the product.
- This is the classic **one-to-many** pattern: one product → many images.

```python
    return {"message": "Product and multiple images added successfully!"}
```

- After all images are processed, return a success message.

6. PATCH `/products/{product_id}` — Update a Product (admin only)

```python
@router.patch("/{product_id}")
async def update_product(
    product_id: str,
    name: Optional[str] = Form(None),
    price: Optional[float] = Form(None),
    brand: Optional[str] = Form(None),
    description: Optional[str] = Form(None),
    color: Optional[str] = Form(None),
    stock_quantity: Optional[int] = Form(None),
    admin_user = Depends(require_admin)
):
```

- `@router.patch("/{product_id}")` → handles `PATCH /products/<id>`.
- `{product_id}` is a **path parameter** — FastAPI extracts it from the URL and passes it in.
- All fields are optional form fields — send only what you want to change.
- Still admin-only.

```python
    update_data = {}
    if name: update_data["name"] = name
    if price: update_data["price"] = price
    if brand: update_data["brand"] = brand
    if description: update_data["description"] = description
    if color: update_data["color"] = color
    if stock_quantity is not None: update_data["stock_quantity"] = stock_quantity
```

- Build a dict containing **only** the fields the user actually sent.
- This is crucial: if we always set every field, we'd accidentally overwrite existing data with `None`.
- Note: `stock_quantity is not None` (not `if stock_quantity`) because `0` is a valid quantity but is falsy.

```python
    if not update_data:
        return {"error": "No new data provided"}
```

- If nothing was sent, don't waste a DB call.

```python
    supabase.table("products").update(update_data).eq("id", product_id).execute()
    return {"message": f"Product {product_id} updated perfectly!"}
```

- `.update(...)` → update fields in `update_data`.
- `.eq("id", product_id)` → only the row matching this ID.
- Return a confirmation message.

7. DELETE `/products/{product_id}` — Delete a Product (admin only)

```python
@router.delete("/{product_id}")
async def delete_product(
    product_id: str,
    admin_user = Depends(require_admin)
):
```

- Handles `DELETE /products/<id>`.
- Admin-only.

```python
    # Note: To avoid paying for dead storage, you should ideally write a quick loop her
e
    # to fetch the image URLs and delete them from the Supabase storage bucket first!
```

- **Very important comment**: right now this only deletes the DB row. But the **uploaded images remain in storage** — you'd keep paying for them. A production version would loop over `product_images`, delete the storage files, then delete the DB rows.

```python
    supabase.table("products").delete().eq("id", product_id).execute()
    return {"message": f"Product {product_id} is completely gone."}
```

- Deletes the product row.
- (If `product_images` has a `FOREIGN KEY` with `ON DELETE CASCADE`, its rows get deleted automatically; otherwise, they'd be orphaned.)

8. GET `/products/{product_id}` — Get One Product + Its Images

```python
@router.get("/{product_id}")
async def get_single_product(product_id: str):
```

- Handles `GET /products/<id>`.
- **No auth required** — anyone can view a product.

```python
    response = supabase.table("products").select("*, product_images(image_url)").eq("id", product_id).execute()
```

This is the interesting part:

- `select("*, product_images(image_url)")` → fetch:
  - All product columns (`*`).
  - Plus a **nested relation** — the related rows from `product_images`, returning just their `image_url`.
- The `product_images(image_url)` syntax uses Supabase's **foreign-key relationship** — it knows `product_images.product_id` points back to `products.id`.
- Result is one product object that contains an array like `"product_images": [{"image_url": "..."}, ...]`.

```python
    if not response.data or len(response.data) == 0:
        raise HTTPException(status_code=404, detail="Product not found")
```

- If nothing came back, return a proper `404 Not Found`.

```python
    return {"data": response.data[0]}
```

- Return the single product object.

9. Big Picture Summary

Route	Method	Who	Purpose
`/products/`	GET	Anyone	List, search, filter, sort products
`/products/`	POST	Admin	Create product + upload images
`/products/{id}`	PATCH	Admin	Update selected fields
`/products/{id}`	DELETE	Admin	Delete product row
`/products/{id}`	GET	Anyone	Get one product + its images

Key concepts used:

1. **Query parameters vs path parameters vs form fields** — three different ways data enters a route.
2. **Dependency injection** (`Depends`) — auth runs before the route.
3. **Defensive parsing** — checking `isinstance` , `len` , `.get()` before trusting data.
4. `.is not None` **vs truthy checks** — critical for numeric fields where `0` is valid.
5. **Chained query building** — you compose the query object, then `.execute()` runs it once.
6. **Supabase storage + DB** — two systems used together: file bytes go to storage, URLs go to the DB.
7. **Nested select** — Supabase can embed related tables in one call (like SQL JOIN).

10. Improvements / Bugs I Noticed

1. `if price:` **in PATCH is a bug** — if an admin tries to set price to `0` , it will be ignored. Should be `if price is not None:` .

2. **Delete leaves orphaned images** — the comment itself flags this. You'd want to clean up storage + `product_images` rows.
3. **No file-type/size validation** on uploads — a malicious user could upload a huge or non-image file.
4. `get_single_product` **returns 404 correctly but PATCH doesn't check** if the row existed — you'd want to verify `response.data` is non-empty before claiming success.
5. **Non-atomic creation** — if image upload fails halfway, you'll have a product with missing/partial images. A transaction or cleanup would be better.
6. `sort_by` **isn't validated** — if a client passes `sort_by=password`, Supabase might error. Safer to whitelist allowed columns.

Would you like me to:

- Walk through how **auth (** `require_admin` **) and** `get_current_user` likely work behind the scenes, or
- Show what a **cleaned-up production version** of this file would look like?



reviews.py
PY 2.91KB

Full Line-by-Line Explanation of `reviews.py`

This file defines the **Reviews API** for an e-commerce backend. It handles:

1. **Adding a review** (only if the user actually bought the product).
2. **Listing all reviews** for a given product.

It uses **FastAPI** (web framework), **Pydantic** (validation), and **Supabase** (database).

Let's go through it step by step.

1. Imports

```python
from fastapi import APIRouter, HTTPException, Depends
from pydantic import BaseModel, Field
from app.database import supabase
from app.dependencies import get_current_user
```

Import	Purpose
`APIRouter`	A mini-app for grouping related routes (all review routes).
`HTTPException`	Returns HTTP errors (like `403 Forbidden`).
`Depends`	FastAPI dependency injection — runs a helper function first (here: auth).
`BaseModel`	Pydantic base class for defining the shape of incoming JSON.
`Field`	Adds extra validation rules (like "between 1 and 5").
`supabase`	The database client.
`get_current_user`	Custom function that verifies the login token and returns the user.

2. Create the Router

```python
router = APIRouter(prefix="/reviews", tags=["Reviews"])
```

- Creates a route group.
- `prefix="/reviews"` → all routes here begin with `/reviews`.
- `tags=["Reviews"]` → label shown in the auto-generated API docs.

3. Define the Request Shape (Schema)

```python
class ReviewCreate(BaseModel):
    product_id: str
    rating: int = Field(..., ge=1, le=5)
    comment: str
```

- `class ReviewCreate(BaseModel):` → defines what a "create review" request body must look like.
- `product_id: str` → required string (the product being reviewed).
- `rating: int = Field(..., ge=1, le=5)`
  - `...` means **required** (no default).
  - `ge=1` → "greater than or equal to 1".
  - `le=5` → "less than or equal to 5".
  - So ratings must be `1, 2, 3, 4,` or `5`. FastAPI will auto-reject anything else with a **422 error**.
- `comment: str` → required string.

FastAPI automatically:

- Parses incoming JSON.
- Validates it matches this shape.
- Rejects invalid data with a helpful error.

4. POST `/reviews/` — Add a Review (logged-in users only)

```python
@router.post("/")
def add_review(payload: ReviewCreate, current_user = Depends(get_current_user)):
```

- `@router.post("/")` → handles `POST /reviews/`.
- `payload: ReviewCreate` → FastAPI parses the JSON body into a `ReviewCreate` object.
- `current_user = Depends(get_current_user)` → **auth required**. Before this function runs, the user's token is verified. If invalid → error. If valid → the user object is passed in.

Step 1 — Check if the user actually bought this product

```python
    try:
        orders_res = supabase.table("orders").select("items").eq("user_id", current_use
r.id).execute()
```

- Query the `orders` table.
- `.select("items")` → only fetch the `items` column (each order's product list).
- `.eq("user_id", current_user.id)` → only orders belonging to the logged-in user.
- `.execute()` → run the query.

The intent: **only buyers can review** (prevents fake reviews from people who never bought anything).

Step 2 — Loop through orders to find the product

```python
        has_purchased = False
```

- A flag to track whether we've found the product in any order.

```python
        if orders_res.data:
            for order in orders_res.data:
                if not isinstance(order, dict):
                    continue
```

- `if orders_res.data:` → only loop if there are orders.
- `for order in orders_res.data:` → iterate through each order.
- `if not isinstance(order, dict): continue` → **defensive check**: skip anything that isn't a dictionary (in case the DB returns something odd).

python

```
                items = order.get("items") or []
                if not isinstance(items, list):
                    continue
```

- `order.get("items")` → safely get the `items` field. Returns `None` if missing.
- `or []` → if it's `None` (or another falsy value), use an empty list instead.
- `if not isinstance(items, list): continue` → skip if items isn't a list (defensive again).

python

```
                if any(
                    isinstance(item, dict)
                    and (item.get("id") == payload.product_id or item.get("product_id")
== payload.product_id)
                    for item in items
                ):
                    has_purchased = True
                    break
```

This is the trickiest part. Let's unpack it:

- `any(...)` → returns `True` if **any** item in the iterable matches the condition.
- The condition for each `item`:
  - `isinstance(item, dict)` → it's a dict (not a string or None).
  - **AND** either:
    - `item.get("id") == payload.product_id` (product stored under key `"id"`), **or**
    - `item.get("product_id") == payload.product_id` (product stored under key `"product_id"`).
- The `for item in items` is a **generator expression** inside `any(...)`.

- If any item matches → `has_purchased = True` and `break` out of the outer loop.

> **Why the double check?** Because order items may store the product identifier under different key names depending on how they were saved. This makes the check robust.

Step 3 — Reject if not purchased

```python
        if not has_purchased:
            raise HTTPException(
                status_code=403,
                detail="Verified buyer check failed. You can only review products you h
ave purchased."
            )
```

- If we never found the product in the user's orders, return **403 Forbidden** with a clear message.

Step 4 — Insert the review

```python
        response = supabase.table("reviews").insert({
            "product_id": payload.product_id,
            "user_id": current_user.id,
            "rating": payload.rating,
            "comment": payload.comment,
        }).execute()
        return {"success": True, "message": "Review added successfully", "data": respon
se.data}
```

- Insert a new row into the `reviews` table with:
  - The product being reviewed.
  - The logged-in user's ID (**never trust a client-supplied user_id!**).
  - The rating and comment.
- Return a success response with the inserted row.

Step 5 — Exception handling

```
python

    except HTTPException as he:
        raise he
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e))
```

- `except HTTPException as he: raise he` → if we raised our own `403`, re-raise it *unchanged* so the client sees the correct status code. Without this, it would fall into the generic `Exception` clause and get converted to a `400`.
- `except Exception as e:` → catch any other error (DB failure, etc.) and return `400 Bad Request` with the error message.

> **Note**: A `403` is typically used for "you're logged in but not allowed." Here it fits ("you didn't buy this product"). A `400` for the generic case works, but a `500` would be more standard for unexpected server errors.

5. GET `/reviews/{product_id}` — List Reviews for a Product

```
python

@router.get("/{product_id}")
def get_product_reviews(product_id: str):
```

- Handles `GET /reviews/<product_id>`.
- `{product_id}` is a **path parameter** — FastAPI extracts it and passes it in.
- **No auth required** — anyone can read reviews.

```
python

    try:
        response = supabase.table("reviews").select("*").eq("product_id", product_id).e
xecute()
        return {"success": True, "data": response.data}
```

```
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e))
```

- Query the `reviews` table.
- `.select("*")` → all columns.
- `.eq("product_id", product_id)` → only rows for this product.
- `.execute()` → run it.
- Return `{"success": True, "data": [...]}`.
- On error, return `400` with the message.

6. Big Picture Summary

Route	Method	Who	Purpose
`/reviews/`	POST	Logged-in users	Add a review (must have purchased the product)
`/reviews/{product_id}`	GET	Anyone	List all reviews for a product

Key concepts used:

1. **Pydantic `Field` validation** — enforces rating is between 1 and 5 automatically.
2. `Depends(get_current_user)` — authenticates the request before the function runs.
3. **"Verified buyer" pattern** — enforce business rules server-side, not just in the frontend.
4. **Defensive data parsing** — `isinstance`, `.get(...) or []` — protects against malformed data.
5. **Generator expressions inside `any()`** — a compact way to ask "does any item match?".
6. `except HTTPException: raise` — preserves intentional HTTP errors before the generic handler catches them.

7. **Never trust client-supplied IDs** — always set `user_id` from the authenticated user.

7. Bugs / Improvements to Consider

1. `rating: int = Field(..., ge=1, le=5)` **is great** — but a comment length limit (e.g., `max_length=1000`) would be smart too.
2. **Only checks the *order list*, not whether the order was actually *completed*** — the comment says "completed orders," but no `status` filter is applied. You'd probably want `.eq("status", "completed")` on the orders query.
3. **No duplicate review prevention** — a user can spam multiple reviews for the same product. You might want a unique constraint or an explicit check.
4. **No pagination** on `get_product_reviews` — a product with 10,000 reviews would be returned all at once.
5. **Generic** `400` **for server errors** — using `500` would be more semantically correct for unexpected DB failures.
6. **The** `except HTTPException: raise` **pattern** is correct but slightly unusual — most people would just let FastAPI handle `HTTPException` without a broad `try/except` around it.
7. **No** `created_at` **/** `updated_at` returned in a specific order — you'd likely want to sort reviews by date, newest first: `.order("created_at", desc=True)` .

Would you like me to:

- Show how to **fix the "completed order" check + prevent duplicate reviews**, or
- Explain how `get_current_user` likely works internally (JWT → user lookup)?



upload.py
PY 1.51KB

Full Line-by-Line Explanation of `upload.py`

This file defines a single API endpoint for **uploading a profile picture**. It uses **FastAPI**

(web framework), **Supabase Storage** (file storage), and **Pydantic-independent** native Python modules for file-type detection.

Let's break it down line by line.

1. Imports

```python
from fastapi import APIRouter, HTTPException, UploadFile, File, Depends
from app.database import supabase
from app.dependencies import get_current_user
import uuid
import mimetypes
```

Import	Purpose
`APIRouter`	A mini-app for grouping related routes (here: all upload routes).
`HTTPException`	Returns HTTP errors (e.g., `400 Bad Request`).
`UploadFile`	The type used when a client uploads a file.
`File`	Says the parameter comes from a **multipart file upload**.
`Depends`	FastAPI dependency injection — runs auth first.
`supabase`	The database + storage client.
`get_current_user`	Custom auth function that verifies the token and returns the user.
`uuid`	Generates universally-unique IDs (used in filenames).
`mimetypes`	Python's built-in module for guessing file extensions from MIME types (like `image/png` → `.png`).

2. Create the Router

```python
router = APIRouter(prefix="/upload", tags=["Uploads"])
```

- `prefix="/upload"` → all routes here begin with `/upload`.
- `tags=["Uploads"]` → groups them under "Uploads" in the auto-generated API docs.

3. POST `/upload/profile-picture` — Upload a Profile Picture

```python
@router.post("/profile-picture")
async def upload_profile_picture(file: UploadFile = File(...), current_user = Depends(get_current_user)):
```

- `@router.post("/profile-picture")` → handles `POST /upload/profile-picture`.
- `async def` → asynchronous because file upload is I/O.
- `file: UploadFile = File(...)` → a required uploaded file (`...` means "required").
- `current_user = Depends(get_current_user)` → auth required. If the token is invalid, the endpoint never runs.

The Docstring

```python
    """
    Receives an image, generates a secure unique name,
    and uploads it directly to Supabase Storage.
    """
```

- A docstring (documentation string).

- FastAPI uses it to auto-generate the endpoint's description in the docs.

Step 1 — Read the raw file data

```python
    try:
        file_bytes = await file.read()
```

- `await file.read()` → reads the entire file into memory as raw **bytes**.
- `await` because reading the incoming stream is asynchronous.
- Now `file_bytes` contains the binary image data.

Step 2 — Guess the file extension securely

```python
        file_extension = mimetypes.guess_extension(file.content_type or ".jpg")
```

- `file.content_type` → the MIME type the client declared, like `"image/png"`.
- `or ".jpg"` → if `content_type` is missing/empty, use `".jpg"` as a fallback.
- `mimetypes.guess_extension(...)` → converts a MIME type to a file extension:
  - `"image/png"` → `".png"`
  - `"image/jpeg"` → `".jpg"`
  - `"image/gif"` → `".gif"`
- **Why "securely"?** Because it **doesn't trust the filename** the client sent. A user could name a file `hack.php` or `virus.exe` — but since we derive the extension from the MIME type, that risk is avoided.

Step 3 — Build a unique, user-scoped path

```python
        unique_filename = f"{current_user.id}/{uuid.uuid4()}{file_extension}"
```

- The path is: `<user_id>/<random-uuid>.<ext>` — e.g., `"a3f...c9/b1d8...e4.png"`.
- **Two clever things here:**

  1. **User folder** (`current_user.id/`) — groups each user's uploads into their own folder. This matches a common Supabase storage policy like: *"a user can only write into a folder named after their own user id."*

  2. **UUID filename** — prevents two users (or the same user) from overwriting each other's files. If everyone uploaded `avatar.png`, they'd collide; using a UUID makes each filename unique.

Step 4 — Upload to Supabase Storage

```python
        supabase.storage.from_("profile-assets").upload(
            path=unique_filename,
            file=file_bytes,
            file_options={"cache-control": "3600", "content-type": file.content_type or "image/jpeg"}
        )
```

- `supabase.storage.from_("profile-assets")` → picks the `profile-assets` **bucket** (a storage container in Supabase).
- `.upload(...)` → uploads the file:
  - `path=unique_filename` → where in the bucket the file should live.
  - `file=file_bytes` → the raw bytes we read earlier.
  - `file_options=...` → HTTP metadata:
    - `"cache-control": "3600"` → browsers/CDNs may cache this image for **3600 seconds (1 hour)**. After that, they'll re-fetch it. Speeds up repeat views.
    - `"content-type": file.content_type or "image/jpeg"` → tells browsers how to render it (`image/png`, `image/jpeg`, etc.). Falls back to JPEG if missing.

Step 5 — Get the public URL

```python
```

```
        public_url = supabase.storage.from_("profile-assets").get_public_url(unique_fil
ename)
        return {"success": True, "url": public_url}
```

- `get_public_url(...)` → builds the public URL that a browser can load, e.g.:

```
text

https://<project>.supabase.co/storage/v1/object/public/profile-assets/<user_id>/<uuid
>.png
```

- Return `{"success": True, "url": ...}` so the frontend can display it (or save it into the user's profile).

Step 6 — Error Handling

```
python

    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e))
```

- If anything fails (bad file, storage error, etc.), return `400 Bad Request` with the error message.

4. Big Picture Summary

Route	Method	Who	Purpose
`/upload/profile-picture`	POST	Logged-in user	Upload an image, store it in Supabase, return a public URL

Key concepts used:

1. `UploadFile` + `File(...)` — FastAPI handles the multipart form parsing for you.

2. `mimetypes.guess_extension` — safe, server-side extension detection based on MIME type rather than trusting the client's filename.
3. **UUID filenames** — avoid collisions.
4. **User-scoped folders** — enables a storage access policy per user.
5. `file_options` — sets HTTP cache + content type so browsers serve the image efficiently.
6. **Returning the public URL** — the DB typically stores only this URL, not the binary file.

5. Things You Could Improve

1. **No size limit** — a malicious client could upload a 2 GB file and exhaust server memory (since you read it all with `file.read()`). FastAPI allows setting a max request size, or you could stream in chunks.
2. **No content-type whitelist** — a user could upload `application/pdf` or `text/html` and (if the bucket is public and served inline) potentially do XSS. You should check that `file.content_type` starts with `"image/"`.
3. **No "old picture" cleanup** — every upload adds a new file. Old avatars stay in storage forever, costing money. You might delete the previous one after a successful upload.
4. **Generic `400` for all errors** — a `500` would be more accurate for unexpected server-side failures. Also, `str(e)` can leak internal details; sanitize if that matters.
5. **No response with the full user object** — you might want to automatically update the `profiles.avatar_url` after upload so the frontend doesn't need a second call.
6. `mimetypes.guess_extension` **can return** `None` — if the MIME type is unknown, `file_extension` becomes `None`, producing a path ending in `None`. Add: `file_extension = file_extension or ".jpg"`.

Would you like me to:

- Show a **hardened version** with size limit + MIME whitelist + old-file cleanup, or
- Explain how Supabase **storage policies** would typically be set up so users can only

write into their own folder?

wishlist.py
PY 1.79KB

Full Line-by-Line Explanation of `wishlist.py`

This file defines the **Wishlist API** for an e-commerce backend. A wishlist is a personal list of products a user wants to save for later.

It supports three actions:

1. **Add** a product to the wishlist.
2. **View** the current user's wishlist.
3. **Remove** a product from the wishlist.

It uses **FastAPI**, **Pydantic**, and **Supabase**.

1. Imports

```python
from fastapi import APIRouter, Depends, HTTPException
from pydantic import BaseModel

from app.database import supabase
from app.dependencies import get_current_user
```

Import	Purpose
`APIRouter`	Groups related routes (all wishlist routes).
`Depends`	FastAPI dependency injection — runs auth first.
`HTTPException`	Returns HTTP errors (e.g., `404 Not Found`).
`BaseModel`	Pydantic base class for defining the incoming JSON shape.

`supabase`	The database client.
`get_current_user`	Custom auth function that verifies the token and returns the user.

> Notice the blank line between `pydantic` and `app.*` imports — just a style choice to separate third-party imports from project imports.

2. Create the Router

```python
router = APIRouter(prefix="/wishlist", tags=["Wishlist"])
```

- `prefix="/wishlist"` → all routes here begin with `/wishlist`.
- `tags=["Wishlist"]` → groups them under "Wishlist" in the auto-generated docs.

3. Define the Request Shape (Schema)

```python
class WishlistCreate(BaseModel):
    product_id: str
```

- Defines what a "add to wishlist" request body looks like.
- Only `product_id` is needed — the user ID comes from the auth token, never from the client.

> **Security note:** The client doesn't send `user_id`. That's intentional — trusting the client for user IDs would let a malicious user add items to someone else's wishlist.

4. POST `/wishlist/` — Add to Wishlist

4. POST /wishlist/ — Add to Wishlist

```python
@router.post("/")
def add_to_wishlist(item: WishlistCreate, current_user=Depends(get_current_user)):
```

- `@router.post("/")` → handles `POST /wishlist/`.
- `item: WishlistCreate` → FastAPI parses the JSON body into this object.
- `current_user = Depends(get_current_user)` → auth required.

Body

```python
    try:
        response = supabase.table("wishlists").insert({
            "user_id": current_user.id,
            "product_id": item.product_id,
        }).execute()
        return {"success": True, "data": response.data}
```

- Insert a row into the `wishlists` table.
- `"user_id": current_user.id` → **always take from the auth token**, never the request body.
- `"product_id": item.product_id` → the product being saved.
- `.execute()` → run the insert.
- Return the created row (usually includes `id`, `user_id`, `product_id`, `created_at`).

```python
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e))
```

- On any error (e.g., product doesn't exist, duplicate entry, etc.), return `400 Bad Request`.

> **Note:** If there's a UNIQUE constraint on `(user_id, product_id)`, trying to add the same

> **Note:** If there's a UNIQUE constraint on `(user_id, product_id)`, trying to add the same product twice would fail here. Good — but the error message might be a raw DB error, so you might want to catch that specifically and return something friendlier like `"Already in wishlist"`.

5. GET `/wishlist/` — View My Wishlist

```python
@router.get("/")
def get_my_wishlist(current_user=Depends(get_current_user)):
```

- `@router.get("/")` → handles `GET /wishlist/`.
- Auth required.

Body

```python
    try:
        response = (
            supabase.table("wishlists")
            .select("id, created_at, products(*)")
            .eq("user_id", current_user.id)
            .execute()
        )
        return {"success": True, "data": response.data}
```

This is the interesting part. Let's unpack the `.select(...)`:

- `"id, created_at, products(*)"` → fetch:
  - The wishlist row's `id`.
  - The wishlist row's `created_at`.
  - **A nested `products` object** containing **all columns** (`*`) of the linked product.

This uses Supabase's **foreign-key relationship embedding** — Supabase sees that

`wishlists.product_id` references `products.id` and automatically joins the two tables. So each returned row looks like:

```
json

{
  "id": "wishlist-row-uuid",
  "created_at": "2025-01-15T12:00:00Z",
  "products": {
    "id": "product-uuid",
    "name": "iPhone 15",
    "price": 999.99,
    "brand": "Apple",
    ...
  }
}
```

- `.eq("user_id", current_user.id)` → only wishlist rows for this user.
- Returns `{"success": True, "data": [...]}`.

Why is this useful? In one query, the frontend gets the whole wishlist AND every product's details — no need for a second API call.

```
python

    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e))
```

- Generic error handling.

6. DELETE `/wishlist/{product_id}` — Remove From Wishlist

```
python

@router.delete("/{product_id}")
def remove_from_wishlist(product_id: str, current_user=Depends(get_current_user)):
    """Remove a product from my wishlist (by product id, for easy toggling)."""
```

- `@router.delete("/{product_id}")` → handles `DELETE /wishlist/<product_id>`

- `@router.delete("/{product_id}")` → handles `DELETE /wishlist/<product_id>`.
- `product_id` is a **path parameter** — FastAPI extracts it from the URL.
- The **docstring** explains why the route uses `product_id` instead of a wishlist row `id`:
  - The frontend can "toggle" wishlist state by product: it knows the product ID already, so it doesn't need to first find the wishlist row's ID.
  - This is a common UX pattern: heart button on a product → click → `POST /wishlist/` or `DELETE /wishlist/{product_id}`.

Body

```python
    try:
        response = (
            supabase.table("wishlists")
            .delete()
            .eq("user_id", current_user.id)
            .eq("product_id", product_id)
            .execute()
        )
```

- Delete from the `wishlists` table.
- **Two `.eq(...)` filters** — this is important:
  - `user_id = current_user.id` → **only delete MY row**. Without this, a malicious user could delete anyone's wishlist items by guessing product IDs.
  - `product_id = product_id` → only delete this product's row.
- `.execute()` → run the delete.

```python
        if not response.data:
            raise HTTPException(status_code=404, detail="Wishlist item not found")
        return {"success": True, "message": "Removed from wishlist"}
```

- `if not response.data:` → if nothing was deleted (either the row didn't exist, or it belonged to someone else), return **404 Not Found**. This is helpful for the frontend,

which can show a proper error.

- Otherwise return a success message.

Exception handling (subtle but important)

```
python

    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e))
```

- **First clause**: `except HTTPException: raise` → if we raised the `404` above, re-raise it as-is, preserving the correct status code. Without this, the broad `except Exception` would catch it and convert it to a `400`.
- **Second clause**: any other failure (e.g., DB error) → return `400`.

> **Note:** In `add_to_wishlist` and `get_my_wishlist`, there's no `except HTTPException: raise` clause because those functions don't raise `HTTPException` themselves. Here we do, so we need to preserve it.

7. Big Picture Summary

Route	Method	Who	Purpose
`/wishlist/`	POST	Logged-in user	Add a product
`/wishlist/`	GET	Logged-in user	List my wishlist (with product details)
`/wishlist/{product_id}`	DELETE	Logged-in user	Remove a product

Key concepts used:

1. **Abstract base class** – `AbstractBaseUser` provides the password hashing and authentication interface.

1. **Always derive `user_id` from the token** — never trust the client.
2. **Nested select** — Supabase auto-joins `products(*)` via the foreign key, so one query returns the full wishlist with product data.
3. **Compound filters on delete** — `.eq("user_id", ...).eq("product_id", ...)` ensures you can only delete your own rows.
4. **Preserving `HTTPException`** — re-raising it before the broad `except Exception` so the correct status code reaches the client.
5. **Toggle-friendly design** — using `product_id` in the URL so the frontend can flip wishlist state directly.

8. Things You Could Improve

1. **Prevent duplicates gracefully** — if there's no UNIQUE constraint on `(user_id, product_id)`, a user could add the same product many times. Add one, or check first.
2. **Friendly duplicate error** — right now adding twice gives a raw DB error message. Catch it and return `"Already in wishlist"`.
3. **Pagination** — `get_my_wishlist` returns everything. A user with 500 items gets all 500 (plus all product data) in one response.
4. **Status codes** — `400` for server errors is imprecise. `500` would be better for unexpected failures.
5. `products(*)` **returns too much** — you could narrow it to `products(id, name, price, image_url)` to shrink the payload.
6. **Return the deleted row** — sometimes the frontend wants confirmation of what was removed. You could return `response.data[0]` in the success message.
7. **No batch remove** — some apps let users clear the whole wishlist at once. Would need a separate endpoint.

Would you like me to:

- Show a **version with duplicate prevention + friendlier errors**, or

- Show a **version with duplicate prevention + friendlier errors**, or
- Explain how the `products(*)` nested select works under the hood (foreign keys + Supabase PostgREST)?

payments.py
PY 432B

Full Line-by-Line Explanation of `payments.py`

This file is a **very small** FastAPI router that exposes a single endpoint: the **M-Pesa STK callback URL**. It's how Safaricom (the Kenyan mobile payments provider) notifies your backend after a customer has been prompted to pay.

Let's break it down.

1. Imports

```python
from fastapi import APIRouter, Request
from app.services.mpesa import process_stk_callback
```

Import	Purpose
`APIRouter`	Groups related routes (all payment routes).
`Request`	FastAPI's raw HTTP request object. Gives you access to the body, headers, etc.
`process_stk_callback`	Your own helper function (defined elsewhere, in `app/services/mpesa.py`) that actually handles the callback data.

> **What is STK?** "STK" = **S**im **T**ool**k**it Push. It's the M-Pesa mechanism where Safaricom pushes a PIN prompt to the user's phone (the popup that says "Enter M-Pesa PIN to pay KES 500 to XYZ"). The result of that popup is what Safaricom sends to your **callback URL**.

2. Create the Router

```python
router = APIRouter(prefix="/payments", tags=["payments"])
```

- `prefix="/payments"` → all routes begin with `/payments`.
- `tags=["payments"]` → groups them in the docs.

3. POST `/payments/callback` — The M-Pesa Callback Endpoint

```python
@router.post("/callback")
async def mpesa_callback(request: Request):
```

- `@router.post("/callback")` → handles `POST /payments/callback`.
- Full path is therefore `POST /payments/callback`.
- `async def` → asynchronous, which is appropriate for I/O.
- `request: Request` → unlike the earlier files, we don't declare the body shape with Pydantic. We just take the **raw request** and read its JSON manually.

The Docstring

```python
    """Canonical M-Pesa STK callback endpoint (configured as DARAJA_CALLBACK_URL)."""
```

- "Canonical" = this is the **official / primary** callback URL.
- `DARAJA_CALLBACK_URL` → the environment variable name that your app reads to configure the callback URL when initiating an STK push. "Daraja" is the name of Safaricom's developer portal/API.

Safaricom's developer portal/API.

- Meaning: when you *start* a payment from elsewhere in your app, you tell Safaricom "when it finishes, call me back at `DARAJA_CALLBACK_URL`" — and this endpoint **is** that URL.

Body

```python
    payload = await request.json()
```

- Reads the incoming JSON body from Safaricom.
- `await` because reading the request body is asynchronous.
- `payload` becomes a Python **dict**.

The typical shape of an M-Pesa STK callback looks something like:

```json
{
  "Body": {
    "stkCallback": {
      "MerchantRequestID": "...",
      "CheckoutRequestID": "...",
      "ResultCode": 0,
      "ResultDesc": "The service request is processed successfully.",
      "CallbackMetadata": {
        "Item": [
          {"Name": "Amount", "Value": 500},
          {"Name": "MpesaReceiptNumber", "Value": "ABC123XYZ"},
          {"Name": "TransactionDate", "Value": 20250115120000},
          {"Name": "PhoneNumber", "Value": 254712345678}
        ]
      }
    }
  }
}
```

> **Note:** `ResultCode: 0` in that payload means **the customer paid successfully**. Any other code (e.g. `1032` = cancelled, `1` = insufficient balance) means it failed.

> other code (e.g., `1032` = cancelled, `1` = insufficient balance) means it failed.

```
python

    process_stk_callback(payload)
```

- Hands the parsed dict off to your service-layer function.
- That function is where all the **real logic** lives — likely:
  - Extracting `CheckoutRequestID`, `ResultCode`, `Amount`, `MpesaReceiptNumber`.
  - Finding the matching order/payment row in the DB.
  - If `ResultCode == 0` → marking the payment as **paid**, updating the order status, maybe sending a confirmation SMS/email.
  - If `ResultCode != 0` → marking the payment as **failed** and storing the reason.
- **It's not awaited**, meaning `process_stk_callback` is a **synchronous** function. That's fine for a small routine, but if it does slow work (DB calls, external APIs), you'd want it `async` and awaited — otherwise it blocks the event loop.

```
python

    return {"ResultCode": 0, "ResultDesc": "Accepted"}
```

This is the **response Safaricom expects**. It's not just a convenience:

- Safaricom's servers call your endpoint, and they check the response.
- The convention is: respond with `{"ResultCode": 0, "ResultDesc": "Accepted"}` to say **"I received your callback successfully."**
- If you return anything else, or if your endpoint returns a non-200 status, Safaricom may:
  - Consider the callback **failed**.
  - **Retry** the callback (which can cause duplicate processing if your code isn't idempotent).

> **Important:** This response is separate from the payment's own `ResultCode`. It's just an acknowledgement that your server processed the notification.

4. Big Picture Summary

Route	Method	Who	Purpose
`/payments/callback`	POST	Safaricom (M-Pesa) servers	Receive STK push result notifications

The overall flow (from a user's perspective):

1. User clicks "Pay with M-Pesa" on your app.
2. Your backend calls Safaricom's **STK Push** API → Safaricom sends a PIN prompt to the user's phone.
3. User enters PIN (or cancels).
4. Safaricom processes it and POSTs the **result** to `DARAJA_CALLBACK_URL` → **this endpoint**.
5. This endpoint reads the JSON, passes it to `process_stk_callback`, and acknowledges with `{"ResultCode": 0, "ResultDesc": "Accepted"}`.
6. Your service function updates your DB (order paid / failed / etc.).

Key concepts used:

1. **Webhook-style endpoint** — an external service pushes data to you, not the other way around.
2. **Raw** `Request` — used instead of a Pydantic model because M-Pesa's payload is deeply nested and variable (success vs. failure shapes differ).
3. **Acknowledgment response** — returns the exact shape Safaricom expects to avoid retries.
4. **Separation of concerns** — this file is only a thin "entry point"; all real work is in `process_stk_callback`.
5. **Callback URL via env var (** `DARAJA_CALLBACK_URL` **)** — keeps the URL configurable across dev/staging/production without code changes.

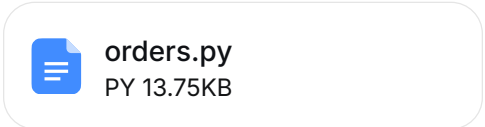
5. Things You Could Improve

5. Things You Could Improve

1. **No signature verification** — Safaricom doesn't provide a signature, but you should at minimum **restrict the source IPs** or use a **secret path segment** (e.g., `/payments/callback/<random-token>`). Otherwise, anyone who learns your URL could POST fake "payment succeeded" payloads.
2. **No error handling** — if `process_stk_callback` throws, FastAPI returns a 500, and Safaricom will **retry** the callback. You probably want a `try/except` around the call so you always return the acknowledgment, and log errors separately.
3. **No logging** — logging the raw payload is crucial for debugging real-world M-Pesa issues (customers say "I paid!" but the DB doesn't show it).
4. **Synchronous service call inside an async route** — if `process_stk_callback` does blocking I/O (DB writes, HTTP calls), it will block the async event loop. Consider making it `async` and `await`ing it, or run it in a thread pool (`await run_in_threadpool(...)`).
5. **Idempotency** — Safaricom can send the same callback more than once. If `process_stk_callback` doesn't check for a previously-processed `CheckoutRequestID`, you could double-credit an order.
6. **The route is unauthenticated** — expected, because Safaricom can't send your auth token. This is why the secret-path approach (#1) matters.
7. **Return type hints** — declaring the response shape (e.g., `-> dict`) and possibly pinning it to a Pydantic response model would improve docs.

Would you like me to:

- Show what a **hardened version** of this endpoint looks like (with secret path + logging + idempotency + safe error handling), or
- Walk through what `process_stk_callback` in `app/services/mpesa.py` **should** do — parsing the callback and updating the DB properly?



Full Line-by-Line Explanation of `orders.py`

This is the **biggest and most important** file so far. It handles the entire **order lifecycle** for the store: checkout, M-Pesa payment, tracking, cancellation, admin status updates, and manual payment verification.

Let me walk through it top to bottom. Because it's long, I'll group things logically and explain **every meaningful line**.

1. Imports

```
python

from __future__ import annotations
```

- A "future" import that makes **type hints lazier** — they're stored as strings instead of being evaluated immediately.
- Why? It lets you write hints like `List[OrderItem]` before `OrderItem` is defined, and it avoids runtime cost for expensive annotations.

```
python

import uuid
from typing import Any, List, Optional
```

- `uuid` → generates unique IDs (used for order IDs).
- `Any` → any type.
- `List` / `Optional` → list of X / X-or-None.

```
python

from fastapi import APIRouter, HTTPException, Request, Depends
from app.database import supabase
from app.routers.daraja_service import DarajaService
from app.utils.email import send_receipt_email
from app.dependencies import get_current_user, require_admin
from app.services.inventory import release_stock_once, restore_order_stock
```

```
from app.services.mpesa import process_stk_callback
from pydantic import BaseModel, EmailStr
```

Import	Purpose
`APIRouter , HTTPException , Request , Depends`	FastAPI essentials.
`supabase`	DB client.
`DarajaService`	Wrapper around Safaricom's Daraja API (initiates STK push, checks transactions).
`send_receipt_email`	Helper that emails the customer a receipt.
`get_current_user , require_admin`	Auth dependencies.
`release_stock_once , restore_order_stock`	Inventory helpers to give stock back to products.
`process_stk_callback`	Handles M-Pesa callback data (shared with `payments.py`).
`BaseModel , EmailStr`	Pydantic — request schemas and email validation.

2. Router + Response/Schema Models

python

```
router = APIRouter(prefix="/orders", tags=["orders"])
```

- All routes here begin with `/orders` .

python

```
class APIResponse(BaseModel):
    success: bool = True
```

```
    data: Optional[Any] = None
    message: Optional[str] = None
```

- A generic response shape. Not used everywhere, but useful for consistency:
  - `success` → always `True` by default.
  - `data` → optional payload.
  - `message` → optional text.

```python
class StatusUpdate(BaseModel):
    status: str  # e.g., "Pending PIN", "Paid", "Processing", "Shipped", "Delivered"
```

- Body for the admin's "update status" endpoint.
- Just one field, plus a comment showing the common status values.

```python
class ManualPaymentVerify(BaseModel):
    receipt_number: str
```

- Body for the admin's "manually verify M-Pesa receipt" endpoint.

3. Helper Functions — Safe Type Coercion

These two functions exist to protect against **bad data in the database** (e.g., `"price": "abc"` or `None`). They convert values to the right type or raise a clear 400 error.

`_coerce_stock_quantity`

```python
def _coerce_stock_quantity(value: object, item_name: str) -> int:
    if isinstance(value, bool) or value is None:
        raise HTTPException(status_code=400, detail=f"Missing or invalid stock quantity
```

```
        raise HTTPException(status_code=400, detail=f"Missing or invalid stock quantity
for {item_name}")

    if not isinstance(value, (str, int, float)):
        raise HTTPException(status_code=400, detail=f"Invalid stock quantity for {item_
name}")

    try:
        return int(value)
    except (TypeError, ValueError):
        raise HTTPException(status_code=400, detail=f"Invalid stock quantity for {item_
name}")
```

- **Why check `bool`?** In Python, `True == 1` and `False == 0`. If someone accidentally stored a boolean, we don't want to silently treat it as a stock number.
- `value is None` → reject.
- `not isinstance(value, (str, int, float))` → allow only numeric-looking values (strings can still be parsed, e.g., `"5"` → `5`).
- `int(value)` → convert to integer. If it fails (e.g., `"abc"`), raise 400.

`_coerce_float`

Same idea, but converts to `float`.

```
python

def _coerce_float(value: object, field_name: str) -> float:
    if isinstance(value, bool) or value is None:
        raise HTTPException(status_code=400, detail=f"Missing or invalid {field_name}")
    if not isinstance(value, (str, int, float)):
        raise HTTPException(status_code=400, detail=f"Invalid {field_name}")
    try:
        return float(value)
    except (TypeError, ValueError):
        raise HTTPException(status_code=400, detail=f"Invalid {field_name}")
```

Used later for prices, delivery costs, and order totals.

4. Request Schemas

```python
class OrderItem(BaseModel):
    id: str
    name: str
    price: float
    qty: int
    image: Optional[str] = ""
```

- One item in the cart. The client sends these, but **prices and names are re-validated later** — never trust them.

```python
class CheckoutRequest(BaseModel):
    phone: str
    email: EmailStr
    fulfillment: str
    location_id: str  # Updated to link to delivery_locations table
    delivery_address: str = ""
    payment_method: str
    items: List[OrderItem]
    total: float
    customer_id: Optional[str] = None
```

- The whole checkout payload.
- `EmailStr` → Pydantic validates it's a real-looking email.
- `total` and item prices are accepted but **ignored** server-side (recomputed).

5. GET `/orders/me` — My Orders

```python
@router.get("/me")
def get_my_orders(current_user = Depends(get_current_user)):
    """Fetch orders for the currently authenticated user."""
```

```
    try:
        response = supabase.table("orders").select("*").eq("user_id", current_user.i
d).order("created_at", desc=True).execute()
        return {"success": True, "data": response.data}
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e))
```

- Auth required.
- Fetch all columns from `orders` where `user_id` matches the logged-in user.
- Sort by `created_at` **descending** — newest first.
- Return.

6. POST `/orders/{order_id}/cancel` — Customer Cancels Own Order

python

```
@router.post("/{order_id}/cancel")
def cancel_my_order(order_id: str, current_user = Depends(get_current_user)):
    """Customer cancels their own order; held stock is restored (idempotent)."""
```

- Path parameter `order_id` . Auth required.
- Docstring says "idempotent" — calling it twice has the same effect as once.

python

```
        order_res = (
            supabase.table("orders")
            .select("*")
            .eq("id", order_id)
            .eq("user_id", current_user.id)
            .execute()
        )
        if not order_res.data or not isinstance(order_res.data[0], dict):
            raise HTTPException(status_code=404, detail="Order not found")
```

- **Two** `.eq` **filters** → you can only fetch orders that belong to you. This prevents a

- **Two `.eq` filters** → you can only fetch orders that belong to you. This prevents a customer from cancelling other customers' orders.
- If no result → 404.

```
python

        order = order_res.data[0]
        if order.get("status") in {"Shipped", "Delivered", "Cancelled"}:
            raise HTTPException(status_code=400, detail="Order can no longer be cancell
ed")
```

- Once shipped, delivered, or already cancelled, cancellation is blocked.

```
python

        if not release_stock_once(order, "Cancelled"):
            # Not holding stock (e.g. already Paid) — just mark it cancelled.
            supabase.table("orders").update({"status": "Cancelled"}).eq("id", order_i
d).execute()
```

- `release_stock_once(order, "Cancelled")` → puts stock back into products **once**, and marks the order "Cancelled". Returns `True` if it did anything, `False` if stock was already released (or not held).
- If stock wasn't held (e.g., the order was already paid and stock treated as sold), just set the status to "Cancelled".

```
python

        return {"success": True, "message": "Order cancelled"}
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e))
```

- Preserve our own `HTTPException` s, wrap everything else as 400.

7. GET `/orders/user/{user_id}` — Admin: One User's Orders

```python
@router.get("/user/{user_id}")
def get_user_orders(user_id: str, _admin_user = Depends(require_admin)):
    """Admin endpoint to fetch a specific user's orders."""
    try:
        response = supabase.table("orders").select("*").eq("user_id", user_id).execute
()
        return {"success": True, "data": response.data}
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e))
```

- `require_admin` → only admins.
- `_admin_user` (underscore prefix) → convention for "we need the dependency to run but don't use the value".

8. GET `/orders/all` — Admin: All Orders

```python
@router.get("/all")
def get_all_orders(admin_user = Depends(require_admin)):
    """Admin endpoint to fetch all orders for the store."""
    try:
        response = supabase.table("orders").select("*").order("created_at", desc=Tru
e).execute()
        return {"success": True, "data": response.data}
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e))
```

- Same as `/me`, but no user filter — all orders.
- Sorted newest first.

> ⚠️ **Ordering caveat:** FastAPI matches routes in the order they're declared. `/orders/all` must be declared **before** any catch-all like `/orders/{order_id}` — luckily, that pattern isn't used here (the dynamic routes have suffixes like `/cancel` and `/track/...`), so we're fine.

9. POST `/orders/checkout` — The Big One

python

```
@router.post("/checkout", status_code=201)
async def process_checkout(payload: CheckoutRequest, current_user = Depends(get_current
_user)):
```

- `status_code=201` → "Created", the correct HTTP code for creating an order.
- Async, because we call an external API (Daraja).

Step 0 — Validate Delivery Location

python

```
        loc_res = supabase.table("delivery_locations").select("name, cost, active").eq
("id", payload.location_id).single().execute()
        delivery_location = loc_res.data
        if not isinstance(delivery_location, dict):
            raise HTTPException(status_code=400, detail="Invalid delivery location")
        if delivery_location.get("active") is False:
            raise HTTPException(status_code=400, detail="Delivery location is currently
unavailable")
```

- Look up the delivery location the client chose.
- `.single()` expects exactly one row.
- Check it's a dict and is `active`.

python

```
        location_name = delivery_location.get("name")
        if not isinstance(location_name, str):
            raise HTTPException(status_code=400, detail="Invalid delivery location")
        validated_delivery_cost = _coerce_float(delivery_location.get("cost"), "deliver
y cost")
```

- Extract and validate `name` and `cost`.

```
python

        if not payload.items:
            raise HTTPException(status_code=400, detail="Cart is empty")
```

- Reject empty carts.

Step 1 — Server-side price + stock validation

```
python

        line_items = []
        items_total = 0.0
        for item in payload.items:
            if item.qty <= 0:
                raise HTTPException(status_code=400, detail=f"Invalid quantity for {ite
m.name}")
```

- Loop over each client-sent item.
- Reject non-positive quantities.

```
python

            prod_res = supabase.table("products").select("name, price, stock_quantit
y").eq("id", item.id).execute()
            if not prod_res.data or not isinstance(prod_res.data[0], dict):
                raise HTTPException(status_code=404, detail=f"Product {item.id} not fou
nd")
```

- Look up the real product.
- If not found → 404.

```
python

            product_row = prod_res.data[0]
            validated_stock = _coerce_stock_quantity(product_row.get("stock_quantity"),
item.name)
            if validated_stock < item.qty:
```

```
                raise HTTPException(status_code=400, detail=f"Insufficient stock for {i
tem.name}")
```

- Coerce stock value, then verify enough stock exists.

python

```
            unit_price = _coerce_float(product_row.get("price"), f"price for {item.nam
e}")
            if unit_price < 0:
                raise HTTPException(status_code=400, detail=f"Invalid price for {item.n
ame}")
```

- Coerce price and reject negatives.

python

```
            items_total += unit_price * item.qty
            line_items.append({
                "id": item.id,
                "name": product_row.get("name") or item.name,
                "price": unit_price,
                "qty": item.qty,
                "image": item.image or "",
            })
```

- Sum line totals.
- Build a **trusted** line item — using DB name/price, not the client's.

python

```
        validated_items_total = round(items_total, 2)
        order_total = round(validated_items_total + validated_delivery_cost, 2)
```

- Round to 2 decimals → currency-friendly.
- Add delivery cost.

python

```
        order_uuid = str(uuid.uuid4())
```

```
short_ref = f"GWV-{order_uuid[:4]}"
is_mpesa = "M-Pesa" in payload.payment_method
```

- Generate a UUID and a short human-readable reference like `GWV-a1b2`.
- `is_mpesa` → a simple string check to branch later.

Step 2 — Persist the order first

python

```
order_record = {
    "id": order_uuid,
    "order_reference": short_ref,
    "checkout_request_id": "",
    "user_id": current_user.id,
    "customer_phone": payload.phone,
    "customer_email": payload.email,
    "fulfillment": payload.fulfillment,
    "location_name": location_name,
    "delivery_cost": validated_delivery_cost,
    "delivery_address": (payload.delivery_address or "").strip(),
    "payment_method": payload.payment_method,
    "total_amount": order_total,
    "status": "Pending PIN" if is_mpesa else "Processing",
    "items": line_items,
}
supabase.table("orders").insert(order_record).execute()
```

- Build the full order row.
- Status logic:
  - **M-Pesa** → `"Pending PIN"` (customer must enter PIN).
  - **Other** (e.g., cash) → `"Processing"`.
- Insert it.

> **Design principle:** Save the order *before* touching stock. If something fails later, you have a record to clean up.

Step 3 — Decrement stock, with rollback

Step 3 — Decrement stock, with rollback

```python
        try:
            for line in line_items:
                prod_res = supabase.table("products").select("stock_quantity").eq("id",
line["id"]).execute()
                if not prod_res.data or not isinstance(prod_res.data[0], dict):
                    raise HTTPException(status_code=404, detail=f"Product {line['id']}
not found")

                current_stock = _coerce_stock_quantity(prod_res.data[0].get("stock_quan
tity"), line["name"])
                new_stock = max(0, current_stock - line["qty"])
                supabase.table("products").update({"stock_quantity": new_stock}).eq("i
d", line["id"]).execute()
        except Exception:
            # Roll back any partial decrements and remove the dangling order
            restore_order_stock(order_record)
            supabase.table("orders").delete().eq("id", order_uuid).execute()
            raise
```

- Loop through items again, re-read each product's stock (fresh value), and decrement.
- `max(0, ...)` → never go negative.
- If **anything** fails mid-loop:
  - `restore_order_stock(order_record)` → re-add stock to all products in the order (even the ones already decremented).
  - Delete the order row we created.
  - Re-raise → error propagates.

> This is a **manual "transaction"** because Supabase's HTTP API doesn't do multi-statement DB transactions easily.

Step 4 — Trigger M-Pesa STK Push

```python
        checkout_request_id = ""
```

```
        if is_mpesa:
            final_total = int(round(order_total))
            if final_total <= 0:
                release_stock_once(order_record, "Payment Failed")
                raise HTTPException(status_code=400, detail="Order total is invalid")
```

- M-Pesa only accepts whole-shilling integer amounts.
- If the total is zero or negative → release stock, mark failed.

python

```
            try:
                daraja = DarajaService()
                stk_response = await daraja.send_stk_push(
                    phone_number=payload.phone,
                    amount=final_total,
                    account_reference=short_ref,
                    transaction_desc=f"Payment for order {short_ref}"
                )
            except Exception:
                # Payment could not be initiated: release stock and mark failed
                release_stock_once(order_record, "Payment Failed")
                raise HTTPException(status_code=502, detail="Failed to initiate M-Pesa
payment. Please try again.")
```

- Call Safaricom via DarajaService.
- If it fails → release stock, mark failed, return `502 Bad Gateway` (because an upstream service failed).

python

```
            checkout_request_id = stk_response.get("CheckoutRequestID", f"sim_{short_re
f}")
            supabase.table("orders").update(
                {"checkout_request_id": checkout_request_id}
            ).eq("id", order_uuid).execute()
```

- Extract the CheckoutRequestID (used later by the callback to match the order).
- Fall back to a fake `sim_<ref>` if Safaricom didn't return one (useful for testing).

- Save it on the order.

```python
        return {
            "status": "success",
            "message": "Order processed. If using M-Pesa, check your phone for the STK prompt.",
            "order_id": short_ref,
            "checkout_request_id": checkout_request_id
        }
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

- Return success info.
- Preserve `HTTPException` s; wrap other errors as 500.

10. GET `/orders/track/{order_ref}` — Public Order Tracking

```python
@router.get("/track/{order_ref}")
def track_order_by_reference(order_ref: str):
    """Allows customers to track their order live using the short reference (e.g., GWV-1234)."""
    try:
        response = supabase.table("orders").select("*").eq("order_reference", order_ref).single().execute()
        if not response.data:
            raise HTTPException(status_code=404, detail="Order not found")
        return {"success": True, "data": response.data}
    except Exception as e:
        raise HTTPException(status_code=404, detail=str(e))
```

- No auth — anyone with the short reference can view the order (like a package tracking number).

- `.single()` → exactly one row.

> ⚠️ **Security note:** Since the reference is only 4 hex chars (`GWV-a1b2` = 65,536 possibilities), it's brute-forceable. Consider making it longer.

11. PATCH `/orders/status/{order_id}` — Admin Updates Status

```python
@router.patch("/status/{order_id}")
def update_order_status(order_id: str, payload: StatusUpdate, _admin_user = Depends(require_admin)):
    """Admin endpoint to push real-time order status updates."""
    try:
        response = supabase.table("orders").update({"status": payload.status}).eq("id", order_id).execute()
        return {"success": True, "message": f"Order status updated to {payload.status}", "data": response.data}
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e))
```

- Admin-only.
- Updates the `status` column with whatever string the admin sent.
- No validation on status values — the admin could send anything.

12. PATCH `/orders/verify-payment/{order_id}` — Admin Manual Verify

```python
@router.patch("/verify-payment/{order_id}")
async def verify_mpesa_payment(order_id: str, payload: ManualPaymentVerify, _admin_user = Depends(require_admin)):
    """Admin endpoint: cross-checks the provided M-Pesa receipt with Daraja."""
```

- Admin-only.

- Docstring says it should cross-check with Daraja, but the actual call is commented out.

```
python

        # daraja = DarajaService()
        # is_valid = await daraja.check_transaction(payload.receipt_number)
```

- **Commented out** — meaning the current behavior trusts the admin's word that the receipt is valid.

```
python

        order_res = supabase.table("orders").select("*").eq("id", order_id).single().ex
ecute()
        order = order_res.data

        if not isinstance(order, dict):
            raise HTTPException(status_code=404, detail="Order not found")
```

- Fetch the order; error if missing.

```
python

        customer_email = order.get("customer_email")
        order_reference = order.get("order_reference")
        total_amount = order.get("total_amount")
        items = order.get("items")

        if not isinstance(customer_email, str) or not customer_email:
            raise HTTPException(status_code=400, detail="Order does not have a valid cu
stomer email")
        if not isinstance(order_reference, str) or not order_reference:
            raise HTTPException(status_code=400, detail="Order reference missing")
        if not isinstance(items, list):
            items = []

        validated_total_amount = _coerce_float(total_amount, "order total")
```

- Extract and validate the fields we need for the receipt email.

python

```
        supabase.table("orders").update({
            "receipt_number": payload.receipt_number,
            "status": "Paid"  # Instantly flips to Paid
        }).eq("id", order_id).execute()

        send_receipt_email(customer_email, order_reference, validated_total_amount, ite
ms)

        return {"success": True, "message": "Payment verified and order approved!"}
```

- Save the receipt number, flip status to "Paid".
- Email the customer a receipt.
- Return success.

python

```
    except HTTPException:
        raise
    except Exception:
        # If it fails, we can log it and set status to "Manual Review"
        raise HTTPException(status_code=400, detail="Invalid receipt or verification fa
iled.")
```

- Note: the comment mentions "set status to Manual Review" but that's not actually implemented.

13. POST `/orders/status-result` — Legacy M-Pesa Callback

python

```
@router.post("/status-result")
async def transaction_status_webhook(request: Request):
    """Legacy alias for the M-Pesa callback. Delegates to the shared service."""
    payload = await request.json()
    process_stk_callback(payload)
    return {"message": "Safaricom update received"}
```

- Same behavior as `payments.py` 's `/payments/callback` .
- Just another URL Safaricom might be configured to hit.
- Delegates to the shared `process_stk_callback` .

> **Note:** Unlike `payments.py` , it responds with `{"message": ...}` instead of `{"ResultCode": 0, "ResultDesc": "Accepted"}` . If Safaricom is configured to hit this endpoint, they might retry — see the recommendation to standardize.

14. Big Picture Summary

Route	Method	Who	Purpose
`/orders/me`	GET	Customer	My orders
`/orders/{order_id}/cancel`	POST	Customer	Cancel my order (restores stock)
`/orders/user/{user_id}`	GET	Admin	One user's orders
`/orders/all`	GET	Admin	All orders
`/orders/checkout`	POST	Customer	Place an order + M-Pesa push
`/orders/track/{order_ref}`	GET	Anyone	Track order by short ref
`/orders/status/{order_id}`	PATCH	Admin	Update status
`/orders/verify-payment/{order_id}`	PATCH	Admin	Manually verify M-Pesa payment
`/orders/status-result`	POST	Safaricom	Legacy callback

Key concepts used:

Key concepts used:

1. **Server-side price/stock validation** — never trust the client's `total` or item prices.
2. **Manual rollback** — if stock decrement fails partway, `restore_order_stock` + delete order.
3. **Idempotency helpers** — `release_stock_once` ensures stock is only released once.
4. **Enum-like status strings** — `"Pending PIN"`, `"Paid"`, `"Shipped"`, etc.
5. `_coerce_*` **helpers** — bulletproof DB value parsing.
6. `except HTTPException: raise` — preserves intentional errors before generic handler.
7. **Order saved before stock touched** — so failures leave a clean record.
8. `status_code=201` — correct HTTP semantics for creation.

15. Bugs / Improvements to Consider

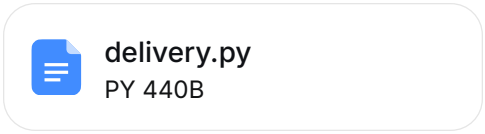
1. **The status value from** `StatusUpdate` **isn't validated** — admins could set `"Pizza"` and break the app. Consider an `Enum`.
2. **Short reference is too short** — `GWV-a1b2` has only ~65k possibilities; easy to brute-force. Use 6–8 chars.
3. **Race condition on stock** — two concurrent checkouts could both read the same "5 left" and both write "3 left", losing an order. True atomic decrement needs a DB-side function or a transaction.
4. **No Stripe/other payment integration** — `is_mpesa` is a string check; other methods just go straight to `"Processing"` without any payment step. Who confirms them?
5. `verify_mpesa_payment` **doesn't actually verify** — the Daraja call is commented out, so any admin (or compromised admin token) could approve anything.
6. `send_receipt_email` **is synchronous** — it blocks the async route. Use a background task or thread pool.
7. **Inconsistent callback response** — `/orders/status-result` should probably return the same acknowledgment shape as `/payments/callback`.
8. **No audit trail** — status changes don't record *who* changed them or *when*. Add `status_updated_by`, `status_updated_at`.

9. `items` **stored as JSONB in the order** — you lose relational integrity (can't easily join). Consider an `order_items` table.
10. `get_all_orders` **has no pagination** — a big store will eventually return thousands of rows in one response.
11. `track_order_by_reference` **catches everything as 404** — even 500-level DB failures become "Order not found", which is misleading.
12. `_coerce_*` **raise** `HTTPException` **directly** — it works, but helper functions raising HTTP errors couples business logic to the web layer. A custom exception would be cleaner.

Would you like me to:

- Diagram the **full M-Pesa checkout flow** end-to-end (checkout → STK → callback → order paid), or
- Show a **hardened** `checkout` with atomic stock handling + background email + enum statuses?

This response is AI-generated, for reference only.



Full Line-by-Line Explanation of `delivery.py`

This is a **very small** file. It exposes one public endpoint: listing the **active delivery locations** the store supports. It's meant to be called by the frontend so the checkout page can show a dropdown like "Nairobi CBD (KES 200)", "Westlands (KES 300)", etc.

Let's walk through it.

1. Imports

```python
from fastapi import APIRouter
```

```
from fastapi import APIRouter
from app.database import supabase
```

Import	Purpose
`APIRouter`	FastAPI's tool for grouping related routes.
`supabase`	The database client (Supabase).

> Notably, **no** `HTTPException` **and no** `Depends` — this endpoint is public, no auth, and (as we'll see) it returns errors in a different style than the other files.

2. Create the Router

```
python

router = APIRouter(prefix="/delivery", tags=["Delivery"])
```

- `prefix="/delivery"` → all routes start with `/delivery`.
- `tags=["Delivery"]` → groups them in the auto-generated docs.

3. GET `/delivery/locations` — List Active Delivery Locations

```
python

@router.get("/locations")
async def get_delivery_locations():
    """Fetch all active delivery locations."""
```

- `@router.get("/locations")` → handles `GET /delivery/locations`.
- Full path = `GET /delivery/locations`.
- `async def` → asynchronous (though the work here could equally be sync).
- No parameters → no auth, no query params, no path params.
- Docstring describes what it does.

Body

```python
    try:
        res = supabase.table("delivery_locations").select("*").eq("active", True).execu
te()
        return {"data": res.data}
```

- `supabase.table("delivery_locations")` → points to the `delivery_locations` table.
- `.select("*")` → fetch all columns (`id` , `name` , `cost` , `active` , maybe `created_at`).
- `.eq("active", True)` → **only return locations marked active**.
  - This is the key business rule: an admin can "soft-disable" a location by flipping `active = False` , and it disappears from the customer UI without deleting the row.
  - This is why orders reference `location_id` and store `location_name` as well — even if a location is later deactivated, past orders keep the correct name.
- `.execute()` → run the query.
- Return `{"data": [...]}` — a simple JSON wrapper.

> **Note:** No `success` flag here, unlike earlier files. This file uses a slightly different response shape (`{"data": ...}` vs. `{"success": True, "data": ...}`). Frontends must account for that inconsistency.

Error Handling

```python
    except Exception as e:
        return {"data": [], "error": str(e)}
```

This is unusual compared to the other files. Let's compare:

File	On error
	raise `HTTPException` → client gets a **4xx/5xx**

`products.py` , `orders.py` , `wishlist.py` , etc.	raise `HTTPException` → client gets a **4xx/5xx** HTTP status.
`delivery.py` (here)	returns `200 OK` with `{"data": [], "error": "..."}` .

Implications:

- **Pro:** The frontend never gets a network error — it always gets a JSON response it can parse. In this case it even gets a safe fallback (`[]`), so the checkout page just shows "no locations" instead of crashing.
- **Con:** A real failure (DB down, wrong table name, permissions) is masked as a **200 success** with empty data. Monitoring/alerting won't notice it. The frontend can't distinguish "no locations exist" from "the server failed" unless it explicitly checks for the `error` key.

> This is a common tradeoff in public, read-only, "always return something usable" endpoints, but it's considered less correct than raising `HTTPException` with a proper `500` .

4. Big Picture Summary

Route	Method	Who	Purpose
`/delivery/locations`	GET	Anyone	List active delivery locations

Where this fits in the app:

- The **checkout** endpoint (`orders.py` → `/orders/checkout`) requires a `location_id` .
- Before showing checkout, the frontend calls `/delivery/locations` to populate the dropdown.
- The customer picks one → `location_id` goes into the checkout payload.
- The backend then re-fetches that location, validates `active` , and uses its `name` and `cost` when creating the order.

So this endpoint and the order's location validation are the two ends of the same feature.

Key concepts used:

1. **Filtering with** `active = True` — soft-disable pattern instead of deleting.
2. **Public endpoint** — no `Depends(get_current_user)`, so no auth.
3. **Fallback response on error** — returns `[]` so the UI keeps working.
4. **Different response shape** — `{"data": ...}` rather than `{"success": True, "data": ...}`, which is inconsistent with the rest of the codebase.

5. Things You Could Improve

1. **Response shape consistency** — every other file returns `{"success": True, "data": ...}`. This one returns `{"data": ...}`. Consider aligning.
2. **Don't swallow errors as 200** — a 500 error should be a 500. The frontend can still show `[]` if it wants, but the backend shouldn't pretend the request "succeeded." Suggested pattern:

```python
from fastapi import HTTPException
try:
    res = supabase.table("delivery_locations").select("*").eq("active", True).execute()
    return {"success": True, "data": res.data}
except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))
```

3. **Limit the columns** — `.select("*")` leaks any future sensitive columns. Better: `.select("id, name, cost")`.
4. **Sort the results** — no `.order(...)`, so the list order from Supabase is unspecified. A stable sort like `.order("cost")` or `.order("name")` gives a predictable UI.
5. **Caching** — this is a great candidate for a short cache (`Cache-Control: max-age=300`), since it changes rarely.

6. `async def` **without** `await` — harmless, but a sync `def` would also work; the sync Supabase client doesn't benefit from async here.
7. **No admin "list all (including inactive) locations" endpoint** — admins probably need to manage inactive ones too; that would be a separate admin-only route.

Would you like me to:

- Write the **admin version** of this file (`GET/POST/PATCH/DELETE` for managing locations), or
- Draft a **small end-to-end picture** of how `/delivery/locations` links to `/orders/checkout` ?

daraja_service.py
PY 3.97KB

Full Line-by-Line Explanation of `daraja_service.py`

This file is the **M-Pesa integration layer**. It wraps Safaricom's "Daraja" API into a Python class, so the rest of the app can call simple methods like `send_stk_push(...)` and `check_transaction(...)` instead of dealing with raw HTTP calls and base64 encoding.

Let's go through it carefully.

1. Imports

```python
import base64
from datetime import datetime
import dotenv
import httpx
from fastapi import HTTPException
import os
import dotenv
```

Import	Purpose

`base64`	Encodes/decodes Base64 — required by Safaricom for the auth header and the STK password.
`datetime`	Gets the current time for the required STK timestamp.
`dotenv`	Loads environment variables from a `.env` file.
`httpx`	Modern async HTTP client — used to make requests to Safaricom.
`HTTPException`	FastAPI error class, used here so failed M-Pesa calls surface as proper HTTP errors.
`os`	Access to environment variables.

> ⚠️ **Redundancy**: `import dotenv` appears **twice**. Harmless (Python just re-imports the same module), but sloppy. Also, the convention is `from dotenv import load_dotenv` — but this file uses `dotenv.load_dotenv()` further down, so this form works too.

2. Load `.env`

```
python

dotenv.load_dotenv()
```

- Reads a `.env` file from the current directory and sets each key/value as an environment variable.
- This is what makes `os.getenv("DARAJA_CONSUMER_KEY")` work.

> **Note:** In modern FastAPI apps, this is often done once in `main.py` or a settings module. Doing it per-file works but means it runs every time this module is imported (idempotent, so it's fine, just slightly redundant).

3. The `DarajaService` Class

```
python

class DarajaService:
```

- A class whose methods each wrap one Daraja endpoint.
- You instantiate it with `DarajaService()` and call methods.

3.1 `__init__` — Load Config

```
python

    def __init__(self):
        self.consumer_key = os.getenv("DARAJA_CONSUMER_KEY", "")
        self.consumer_secret = os.getenv("DARAJA_CONSUMER_SECRET", "")
        self.shortcode = os.getenv("DARAJA_SHORTCODE", "174379")
        self.passkey = os.getenv("DARAJA_PASSKEY", "")
        self.security_credential = os.getenv("DARAJA_SECURITY_CREDENTIAL", "")
        self.callback_url = os.getenv("DARAJA_CALLBACK_URL", "")
        sandbox = os.getenv("DARAJA_IS_SANDBOX", "true").lower() == "true"
        self.base_url = "https://sandbox.safaricom.co.ke" if sandbox else "https://ap
i.safaricom.co.ke"
```

Each line reads an environment variable with a default:

Attribute	Purpose
`consumer_key` / `consumer_secret`	Credentials issued by Safaricom's Daraja portal. Used to get an OAuth access token.
`shortcode`	The M-Pesa "paybill" or "till" number. `174379` is Safaricom's well-known **sandbox** shortcode.
`passkey`	Secret string used to build the STK password.
`security_credential`	Encrypted credential required for certain

`security_credential`	"initiator" APIs (like Transaction Status).
`callback_url`	Where Safaricom will POST the STK result — the `/payments/callback` endpoint from earlier.
`sandbox`	Reads `DARAJA_IS_SANDBOX` (default `"true"`), lowercases it, and compares to `"true"`. Yields a boolean.
`base_url`	**Ternary**: sandbox URL if sandbox, production URL otherwise.

> **Key idea:** all secrets come from environment variables — **never hard-code them in source**. This is what keeps credentials out of Git.

3.2 `get_access_token` — OAuth Token Fetch

```python
    async def get_access_token(self):
        url = f"{self.base_url}/oauth/v1/generate?grant_type=client_credentials"
        auth = base64.b64encode(f"{self.consumer_key}:{self.consumer_secret}".encode
()).decode()
```

- `async def` → this method performs I/O and can be awaited.
- Builds the OAuth URL. Daraja uses **OAuth 2.0 client credentials** flow.
- Base64-encodes `"<key>:<secret>"` → this is the classic **HTTP Basic Auth** header format.
  - `.encode()` → string to bytes (base64 works on bytes).
  - `.decode()` → bytes back to a plain string (needed for the header).

```python
        async with httpx.AsyncClient() as client:
            res = await client.get(url, headers={"Authorization": f"Basic {auth}"})
            if res.status_code != 200:
```

```
            if res.status_code != 200:
                raise HTTPException(status_code=400, detail="Failed to connect to M-Pes
a.")
            return res.json().get("access_token")
```

- `async with httpx.AsyncClient() as client:` → creates an async HTTP client; `async with` ensures it's cleaned up.
- `await client.get(...)` → performs the GET request with the Basic auth header.
- If it's not a `200 OK` → raise a 400 with a friendly message.
- `res.json()` → parse the response body as JSON; it looks like `{"access_token": "...", "expires_in": "3599"}`.
- Return just the token string.

> **Note:** Safaricom actually uses **HTTP Basic** for this endpoint, not Bearer — but the header value is literally `Basic <base64(...)>`, so that's correct. (Don't confuse this with the Bearer token returned later.)

3.3 `send_stk_push` — Trigger the PIN Prompt

python

```
    async def send_stk_push(self, phone_number: str, amount: int, account_reference: st
r, transaction_desc: str):
        token = await self.get_access_token()
        url = f"{self.base_url}/mpesa/stkpush/v1/processrequest"
```

- Get a fresh access token.
- Build the STK push URL.

python

```
        timestamp = datetime.now().strftime("%Y%m%d%H%M%S")
        password = base64.b64encode(f"{self.shortcode}{self.passkey}{timestamp}".encode
()).decode()
```

- Safaricom requires the **timestamp format** `YYYYMMDDHHMMSS`.

- The STK password is famously: `base64(shortcode + passkey + timestamp)`.
- Both are **required** for the STK request — Daraja rejects any mismatch.

```python
        payload = {
            "BusinessShortCode": self.shortcode,
            "Password": password,
            "Timestamp": timestamp,
            "TransactionType": "CustomerPayBillOnline",
            "Amount": amount,
            "PartyA": phone_number,
            "PartyB": self.shortcode,
            "PhoneNumber": phone_number,
            "CallBackURL": self.callback_url,
            "AccountReference": account_reference,
            "TransactionDesc": transaction_desc
        }
```

Field	Meaning
`BusinessShortCode`	The paybill/till receiving money.
`Password`	The base64 password we just built.
`Timestamp`	Same timestamp used for the password — **must match exactly**.
`TransactionType`	`"CustomerPayBillOnline"` = customer pays a paybill. (For "Buy Goods" it'd be `CustomerBuyGoodsOnline`.)
`Amount`	Whole-shilling integer amount.
`PartyA`	The **payer's** phone number.
`PartyB`	The **receiver's** shortcode (same as `BusinessShortCode` for paybill).
`PhoneNumber`	The phone that gets the PIN prompt.
`CallBackURL`	Where the result is POSTed.

`AccountReference`	Shown on the customer's M-Pesa statement — we use the short order ref (`GWV-a1b2`).
`TransactionDesc`	A short description shown in the prompt.

```python
        async with httpx.AsyncClient() as client:
            res = await client.post(url, json=payload, headers={"Authorization": f"Bear
er {token}"})
            data = res.json()
            if res.status_code != 200 or data.get("ResponseCode") not in ("0", 0):
                raise HTTPException(status_code=400, detail=data.get("errorMessage") or
data.get("ResponseDescription") or "STK push failed")
            return data
```

- POST with Bearer token.
- **Two checks**:
  - HTTP status must be 200.
  - `ResponseCode` must be `"0"` or `0` (Daraja sometimes returns strings, sometimes ints — hence the tuple check).
- On failure:
  - Prefer `errorMessage` , else `ResponseDescription` , else a generic fallback.
  - This triple-fallback gives a useful error no matter which shape Daraja returned.
- Return the full response (contains `CheckoutRequestID` , `MerchantRequestID` , etc.).

3.4 `check_transaction` — Verify a Receipt

```python
    async def check_transaction(self, transaction_id: str) -> bool:
        """Queries Safaricom to verify if an M-Pesa receipt is real."""
        access_token = await self.get_access_token()
```

- Given an M-Pesa receipt number, ask Safaricom whether it's real.
- Returns a boolean.

python

```
        # The official Daraja sandbox endpoint for transaction status
        api_url = "https://sandbox.safaricom.co.ke/mpesa/transactionstatus/v1/query"
```

> ⚠️ **Bug**: This URL is **hard-coded to sandbox**. If `DARAJA_IS_SANDBOX=false`, this call would still hit sandbox. It should use `f"{self.base_url}/mpesa/transactionstatus/v1/query"`.

python

```
        headers = {
            "Authorization": f"Bearer {access_token}",
            "Content-Type": "application/json"
        }
```

- Standard Bearer auth, JSON body.

python

```
        payload = {
            "Initiator": "testapi",
            "SecurityCredential": self.security_credential,
            "CommandID": "TransactionStatusQuery",
            "TransactionID": transaction_id,
            "PartyA": self.shortcode,
            "IdentifierType": "4",
            "ResultURL": f"{self.callback_url}/status-result",
            "QueueTimeOutURL": f"{self.callback_url}/status-timeout",
            "Remarks": "Manual receipt verification",
            "Occasion": "Order Checkout"
        }
```

Field	Meaning
`Initiator`	The Daraja username authorized to perform the query. `"testapi"` is the sandbox default.

`SecurityCredential`	Encrypted password tied to the initiator.
`CommandID`	The type of query. `TransactionStatusQuery` = "what's the status of this transaction?"
`TransactionID`	The M-Pesa receipt number we're verifying.
`PartyA`	Who initiates — usually the shortcode.
`IdentifierType`	`"4"` = shortcode (as opposed to phone number `"1"` or org `"2"`).
`ResultURL`	Where Safaricom will POST the result. Here, `/orders/status-result` (the legacy alias).
`QueueTimeOutURL`	Where Safaricom POSTs if the query times out.
`Remarks` / `Occasion`	Free-form strings required by the API.

> **Important design note:** This is an **asynchronous** API. Safaricom doesn't return the result inline — it calls your `ResultURL` later. So this method's immediate return value can only tell you **"the query was accepted"**, not "the transaction is valid."

python

```
        async with httpx.AsyncClient() as client:
            response = await client.post(api_url, json=payload, headers=headers)
            data = response.json()

            # A ResponseCode of "0" means Safaricom accepted the query
            if data.get("ResponseCode") == "0":
                return True

        return False
```

- POST the query.
- If `ResponseCode == "0"`, the query was accepted → return `True`.
- Otherwise `False`.
- Again, this is **not** the same as "the transaction is valid" — the actual result comes via

- Again, this is **not** the same as "the transaction is valid" — the actual result comes via the callback.

4. Big Picture Summary

Method	Purpose	Sync/Async
`__init__`	Load config from env vars, pick base URL by sandbox flag.	sync
`get_access_token`	OAuth: exchange key/secret for a Bearer token.	async
`send_stk_push`	Send a PIN prompt to a customer's phone.	async
`check_transaction`	Ask Safaricom about a receipt (result arrives via callback).	async

Where this fits in the app:

```
text

Customer clicks "Pay"
   → /orders/checkout
      → DarajaService().send_stk_push(...)
         → Safaricom pushes PIN to phone
         → Safaricom POSTs result to /payments/callback
            → process_stk_callback(payload)
               → updates order status
```

Key concepts used:

1. **OAuth 2.0 client credentials flow** — key/secret → access token → Bearer auth.
2. **Base64 Basic auth** for the token endpoint.
3. **base64(shortcode + passkey + timestamp)** — the Daraja STK password recipe.
4. **Env-var config with defaults** — secrets stay out of source; sane fallbacks for

sandbox.

5. **Sandbox vs production via a boolean flag** — one attribute determines which URL family is used.
6. **Async HTTP with** `httpx.AsyncClient` — non-blocking outbound calls.
7. **Defensive response parsing** — `data.get(...)` with tuple checks (`("0", 0)`) and layered fallbacks.
8. **Asynchronous verification model** — `check_transaction` only confirms acceptance, not result.

5. Bugs / Improvements to Consider

1. **Duplicate** `import dotenv` — remove one; prefer `from dotenv import load_dotenv` and call `load_dotenv()` .
2. **Hard-coded sandbox URL in** `check_transaction` — should be `f"{self.base_url}/mpesa/transactionstatus/v1/query"` .
3. `get_access_token` **raises 400 for what is really a 502/503 problem** — the *client* isn't wrong, the upstream is. `502 Bad Gateway` is more accurate.
4. **No caching of the access token** — Daraja tokens last ~1 hour. Every STK push fetches a new one; that's an extra round-trip and you may hit rate limits. Cache it with an expiry.
5. `send_stk_push` **doesn't validate inputs** — no check that `amount > 0` , `phone_number` matches the Kenyan format `2547XXXXXXXX` , or `callback_url` is set. A single misconfigured env var breaks every payment.
6. **No timeout on HTTP calls** — `httpx.AsyncClient()` defaults to 5s, but Safaricom can be slow. Explicitly set `timeout=httpx.Timeout(10.0, connect=5.0)` .
7. **No retry logic** — transient network errors will fail a payment outright. Consider `tenacity` or a simple retry.
8. `check_transaction` **return value is misleading** — its name suggests it verifies the transaction; it actually only verifies the **query was accepted**. Rename to `request_transaction_status` or make it clear in the docstring.
9. **No logging** — you'd want to log every request (minus secrets) and every response for

M-Pesa debugging.

10. `security_credential` **default is** `""` — the Transaction Status API will always fail with an empty value. Fail-fast at startup if it's missing.
11. **Callback URL is used as a base** (`f"{self.callback_url}/status-result"`) — but in `orders.py` the STK callback URL is used directly. This means `DARAJA_CALLBACK_URL` must NOT already include `/payments/callback` . That's a subtle, easy-to-break contract worth documenting.
12. **No phone number normalization** — Safaricom expects `2547XXXXXXXX` . If the client sends `+254...` or `07...` , the push fails. Add a normalizer.

Would you like me to:

- Show a **hardened** `DarajaService` (token caching + retries + phone normalization + timeouts + proper logging), or
- Diagram the **full async callback flow** (STK → result URL → `process_stk_callback` → order updated) end-to-end?

categories.py
PY 597B

Full Line-by-Line Explanation of `categories.py`

This is a **small, focused file** — a single public endpoint that returns the list of **active product categories** (like "Phones", "Accessories", "Chargers"). The frontend uses this to build the store's navigation menu or filter sidebar.

Let's walk through every line.

1. Imports

```python
from fastapi import APIRouter, HTTPException
```

```
from app.database import supabase
```

Import	Purpose
`APIRouter`	FastAPI's tool for grouping routes.
`HTTPException`	FastAPI error class — used here to raise a proper 500.
`supabase`	The database client.

> **Style note:** A blank line separates FastAPI from the app-internal import. This is a common convention — third-party imports first, then a blank line, then project imports.

> **Notable absences:** No `Depends`, no `get_current_user`, no `require_admin`, no `Query`. This endpoint is **public** and takes **no parameters**.

2. Create the Router

```
python

router = APIRouter(prefix="/categories", tags=["Categories"])
```

- `prefix="/categories"` → all routes here start with `/categories`.
- `tags=["Categories"]` → groups them under "Categories" in the auto-generated docs.

3. GET `/categories/` — List Active Categories

```
python

@router.get("/")
def list_active_categories():
    """Public list of active categories for the storefront navigation."""
```

- `@router.get("/")` → handles `GET /categories/`.

- Full path = `GET /categories/` .
- `def` (not `async def`) → this is a **synchronous** function. Fine, because the Supabase sync client blocks anyway; FastAPI will run sync endpoints in a thread pool.
- **No parameters** → no auth, no query params, no path params.
- Docstring: "Public list of active categories for the storefront navigation."
  - **Public** → anyone can call it (customers before login).
  - **storefront navigation** → this is exactly where it's used (menu / filter sidebar).

Body

```python
    try:
        response = (
            supabase.table("categories")
            .select("*")
            .eq("active", True)
            .order("name")
            .execute()
        )
        return {"success": True, "data": response.data}
```

Let's unpack the chained query:

- `supabase.table("categories")` → points at the `categories` table.
- `.select("*")` → fetch all columns (e.g., `id` , `name` , `slug` , `image_url` , `active` , `created_at`).
- `.eq("active", True)` → **only return active categories**.
  - Same **soft-disable pattern** we saw in `delivery.py` . An admin can retire a category by flipping `active = False` without deleting it — preserving any products that reference it.
- `.order("name")` → sort alphabetically by name. Ascending by default.
  - This is better than sorting client-side — consistent ordering, easier for the frontend.
- `.execute()` → run the query.

- Return `{"success": True, "data": [...]}` — consistent with most of the codebase.

> **Query-building recap:** Each method returns a new query object. **Nothing hits the database until** `.execute()`. This is a fluent API, like SQLAlchemy or Knex.js.

Error Handling

```python
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

- On any failure (DB down, wrong table name, permissions), raise **500 Internal Server Error** with the error message.
- This is the **correct** approach — unlike `delivery.py`, which returned `{"data": [], "error": str(e)}` with a 200. This file doesn't pretend success when something failed.

> **Minor note:** `str(e)` can leak internal details (table names, keys). In a production setting you might log the error and return a generic message like `"Failed to fetch categories"`.

4. Big Picture Summary

Route	Method	Who	Purpose
`/categories/`	GET	Anyone	List active categories alphabetically

Where this fits in the app:

- The **storefront** (home page, category sidebar) calls this before rendering.
- The **checkout** page might not need it — but product filtering likely does.
- **Admin** flows (creating products with a category dropdown) might hit this too — or they'd use a separate admin endpoint that also returns inactive categories.

Comparison with `delivery.py`:

Aspect	`categories.py`	`delivery.py`
Auth	None (public)	None (public)
Filter	`.eq("active", True)`	`.eq("active", True)`
Sort	`.order("name")`	*(none)*
Error	Raise 500	Return `{"data": []}` with 200
Response	`{"success": True, "data": ...}`	`{"data": ...}`

This file is the "more correct" version of the same pattern.

Key concepts used:

1. **Public endpoint** — no `Depends`, no auth.
2. **Soft-disable pattern** — `active = True` filter.
3. **Fluent query building** — `.table().select().eq().order().execute()`.
4. **Proper error signaling** — raises `HTTPException(500)` instead of silently succeeding.
5. **Alphabetical sort** — `.order("name")` for predictable UI.
6. **Sync endpoint** — FastAPI runs it in a thread pool.

5. Things You Could Improve

1. **Add `slug` to the select** — if the frontend builds URLs like `/category/phones`, you need the slug. `.select("*")` includes it, but the frontend would benefit from a clear contract; consider `.select("id, name, slug, image_url")`.
2. **Response caching** — categories change rarely. A `Cache-Control: max-age=300` header would save DB calls.
3. **No admin endpoint here** — the file has only the public one. Admins need list-all (including inactive), create, update, delete. Those belong in this file too, protected by

`require_admin`.

4. `str(e)` **in the error response** — consider logging and returning a generic message.
5. **No pagination** — irrelevant now, but if categories ever number in the hundreds, add `.limit(...)`.
6. **Not** `async def` — harmless, but since the Supabase Python client is synchronous, either style works. Keep consistency within the file (and consider using the **async** Supabase client if you want the endpoint to be truly async).
7. **Alphabetical sort may not match business intent** — some stores want a custom display order (e.g., "Phones" first). That would need a `sort_order` column and `.order("sort_order")` instead of `.order("name")`.

Would you like me to:

- Draft the **matching admin CRUD endpoints** for categories (create / update / deactivate), or
- Do a **cross-file summary** showing all the patterns you've now seen (auth, soft-disable, coercion helpers, callback flow), with a cheat sheet you can keep?



cart.py
PY 3.15KB

Full Line-by-Line Explanation of `cart.py`

This file defines the **Cart API** — the "shopping cart" feature where customers collect items before checking out. It's a **per-user list of products with quantities**, and it's the front half of the checkout flow you saw in `orders.py`.

The file supports 5 operations:

1. **Add** an item (or increment its quantity).
2. **View** my cart.
3. **Update** an item's quantity.
4. **Remove** a single item.

5. **Clear** the entire cart.

It uses **FastAPI**, **Pydantic**, and **Supabase**.

1. Imports

```python
from fastapi import APIRouter, Depends, HTTPException
from pydantic import BaseModel, Field

from app.database import supabase
from app.dependencies import get_current_user
```

Import	Purpose
`APIRouter`	Groups the cart routes.
`Depends`	Runs auth before each route.
`HTTPException`	Returns HTTP errors (400, 404).
`BaseModel`	Pydantic base class for request schemas.
`Field`	Adds validation rules (like `ge=1`).
`supabase`	The database client.
`get_current_user`	Auth dependency.

2. Create the Router

```python
router = APIRouter(prefix="/cart", tags=["Cart"])
```

- `prefix="/cart"` → all routes here start with `/cart`

- `prefix="/cart"` → all routes here start with `/cart`.
- `tags=["Cart"]` → grouped in docs.

3. Request Schemas

```python
class CartItem(BaseModel):
    product_id: str
    quantity: int = 1
```

- Body of "add to cart".
- `quantity` defaults to `1` (a sensible default — clicking "Add" usually means one unit).

```python
class CartItemUpdate(BaseModel):
    quantity: int = Field(..., ge=1)
```

- Body of "update quantity".
- `...` → required (no default).
- `ge=1` → must be **at least 1**. Zero is rejected — to remove an item entirely, the frontend should use `DELETE /cart/{item_id}` instead.
- If the client sends `0`, FastAPI returns **422 Unprocessable Entity** automatically.

> **Notably, there's no `quantity` upper bound** — a client could pass `quantity=9999999`. The update endpoint partially guards this with a stock check, but `add_to_cart` does **not** (see the improvements section).

4. POST `/cart/add` — Add to Cart (or Increment)

```python
@router.post("/add")
async def add_to_cart(item: CartItem, current_user=Depends(get_current_user)):
```

```
async def add_to_cart(item: CartItem, current_user=Depends(get_current_user)):
```

- `@router.post("/add")` → handles `POST /cart/add`.
- `item: CartItem` → parsed from JSON body.
- Auth required.

Body

```python
    try:
        # If the item already exists, increment; otherwise insert a new row.
        res = supabase.table("cart_items").select("*").eq("user_id", current_user.id).e
q("product_id", item.product_id).execute()
```

- Look for an existing row for this user + product.
- The double `.eq(...)` guarantees we only consider **my** cart.

```python
        if res.data:
            new_qty = res.data[0]["quantity"] + item.quantity
            supabase.table("cart_items").update({"quantity": new_qty}).eq("id", res.dat
a[0]["id"]).execute()
        else:
            supabase.table("cart_items").insert({
                "user_id": current_user.id,
                "product_id": item.product_id,
                "quantity": item.quantity,
            }).execute()
```

- **If the item is already in the cart** → add the new quantity to the existing one ("increment").
  - E.g., cart has 2 phones, user adds 3 more → becomes 5.
- **Else** → insert a new row with `user_id` (from the token, never from the body), `product_id`, and `quantity`.

```python
```

```
        return {"message": "Added to cart"}
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e))
```

- Return a simple success message.
- On error → 400. Note: unlike other files, this one doesn't re-raise `HTTPException` first, because it doesn't raise any itself.

> **Design pattern:** "Find-or-create" (a.k.a. "upsert" logic implemented manually). If there's a **UNIQUE constraint** on `(user_id, product_id)` in the DB, this is safe. Without it, concurrent adds could create duplicate rows.

5. GET `/cart/` — View My Cart

python

```
@router.get("/")
async def get_cart(current_user=Depends(get_current_user)):
    res = supabase.table("cart_items").select("*, products(*)").eq("user_id", current_u
ser.id).execute()
    return {"data": res.data}
```

- Fetch all `cart_items` rows where `user_id` = mine.
- `"*, products(*)"` → same **nested select** trick we saw in `wishlist.py`. Supabase's foreign-key awareness automatically joins the related `products` table so each cart row includes full product details (name, price, image, etc.):

json

```
{
  "id": "cart-row-uuid",
  "user_id": "...",
  "product_id": "...",
  "quantity": 2,
  "products": {
    "id": "product-uuid",
    "name": "iPhone 15"
```

```
      "name": "iPhone 15",
      "price": 999.99,
      "image_url": "..."
    }
  }
```

- `{"data": ...}` shape (not `{"success": True, "data": ...}`) — inconsistent with most other files but matches `delivery.py`.
- **No try/except** — if the DB errors, FastAPI returns a 500 and the client sees a generic error, without a friendly message.

6. PATCH `/cart/{item_id}` — Update Quantity

```python
@router.patch("/{item_id}")
async def update_cart_item(item_id: str, payload: CartItemUpdate, current_user=Depends(get_current_user)):
    """Set a cart line's quantity (ownership enforced by user_id)."""
```

- Handles `PATCH /cart/<item_id>`.
- `item_id` = the **cart row's id** (not the product id).
- Auth required.
- Docstring highlights the ownership check: **you can only update your own rows**.

Ownership check

```python
    res = (
        supabase.table("cart_items")
        .select("id, product_id")
        .eq("id", item_id)
        .eq("user_id", current_user.id)
        .execute()
    )
    if not res.data:
```

```
        raise HTTPException(status_code=404, detail="Cart item not found")
```

- Two `.eq` filters:
  - `id = item_id` → this specific row.
  - `user_id = current_user.id` → must belong to me.
- If no match (either it doesn't exist, or it's someone else's) → 404.
- This is the **compound filter pattern** again — critical for security.

Stock check

```
python

    # Refuse to add more than what is in stock.
    product_id = res.data[0]["product_id"]
    prod = supabase.table("products").select("stock_quantity").eq("id", product_id).exe
cute()
    if prod.data and isinstance(prod.data[0], dict):
        stock = prod.data[0].get("stock_quantity")
        try:
            if stock is not None and payload.quantity > int(stock):
                raise HTTPException(status_code=400, detail="Not enough stock")
        except (TypeError, ValueError):
            pass
```

- Get the product id from the cart row.
- Fetch the product's stock.
- If stock is known:
  - Convert it to int.
  - If the requested quantity **exceeds** stock → 400.
- **Defensive** `try/except (TypeError, ValueError): pass` → if `stock` isn't a number (e.g., `None`, `"abc"`), silently skip the check rather than crash.
  - This means: **bad stock data → no enforcement**. Safe-ish but silent; the checkout endpoint is the true last line of defense.

> ⚠️ **Important**: this stock check is **advisory**. Between this check and actually placing

> the order, stock could change. The **real** enforcement happens in `orders.py` → `/orders/checkout`, which re-validates and decrements.

The update

```python
    supabase.table("cart_items").update({"quantity": payload.quantity}).eq("id", item_id).eq("user_id", current_user.id).execute()
    return {"message": "Cart updated"}
```

- Update the row — **again with the ownership filter** (`user_id = current_user.id`).
  - This second check is belt-and-suspenders: even if the earlier select passed, we still scope the write.
- Return a confirmation.

> **Note:** This is a **"set"** operation, not an "increment". Sending `quantity: 5` sets it to 5, not "+5". The frontend must send the target absolute value.

7. DELETE `/cart/{item_id}` — Remove One Item

```python
@router.delete("/{item_id}")
async def remove_cart_item(item_id: str, current_user=Depends(get_current_user)):
    res = (
        supabase.table("cart_items")
        .delete()
        .eq("id", item_id)
        .eq("user_id", current_user.id)
        .execute()
    )
    if not res.data:
        raise HTTPException(status_code=404, detail="Cart item not found")
    return {"message": "Removed from cart"}
```

- Delete **my** cart row with this id.

- **Two** `.eq` **filters** → can only delete my own.
- If no row was deleted → 404 (either the row is gone or belongs to someone else).
- Return a confirmation.

8. DELETE `/cart/` — Clear My Cart

```python
@router.delete("/")
async def clear_cart(current_user=Depends(get_current_user)):
    supabase.table("cart_items").delete().eq("user_id", current_user.id).execute()
    return {"message": "Cart cleared"}
```

- Delete **all** cart rows for the logged-in user.
- Only **one filter** here — `user_id = current_user.id` .
- No 404 check: an empty cart is still a valid state to "clear".
- Common use case: after successful checkout, the frontend calls this to empty the cart.

> **Combined with** `orders.py` : In `orders.py` 's `/checkout` , the code currently **doesn't** call `clear_cart` . So the frontend is expected to call `DELETE /cart/` after checkout succeeds. That's a reasonable design but could leave stale cart rows if the frontend forgets.

9. Big Picture Summary

Route	Method	Who	Purpose
`/cart/add`	POST	Logged-in user	Add/increment an item
`/cart/`	GET	Logged-in user	View my cart (with product details)

`/cart/{item_id}`	PATCH	Logged-in user	Set a line's quantity (with stock check)
`/cart/{item_id}`	DELETE	Logged-in user	Remove one line
`/cart/`	DELETE	Logged-in user	Clear my whole cart

Key concepts used:

1. **Per-user isolation** — every query filters on `user_id = current_user.id`.
2. **Compound filters on writes** — `PATCH` and `DELETE` filter by both `id` and `user_id` for safety.
3. **Nested select** — `"*, products(*)"` auto-joins product data in one query.
4. **Find-or-create pattern** — `add_to_cart` checks first, then either increments or inserts.
5. **Pydantic Field validation** — `ge=1` on quantities (but no upper bound, no stock check on add).
6. **Advisory stock check** — the `PATCH` verifies stock, but the **real** enforcement is in checkout.
7. **"Set" vs "increment" semantics** — `PATCH` sets the absolute quantity; `POST /add` increments.

10. Bugs / Improvements to Consider

1. **No stock check on** `add_to_cart` — a user can add 10,000 of a product even if only 3 are in stock. The frontend would only discover this at checkout. Move the same stock check into `add_to_cart`.
2. **Race condition on** `add_to_cart` — two concurrent adds could both see `res.data = []` and both insert → two rows for the same `(user_id, product_id)`. Prevent with a **UNIQUE constraint** and use Supabase's atomic upsert:

```
python
```

```
supabase.rpc("increment_cart_item", {...})  # or
supabase.table("cart_items").upsert({...}, on_conflict="user_id,product_id")
```

3. **Product existence not validated** — nothing prevents adding a `product_id` that doesn't exist. The nested `products(*)` in `get_cart` would return `null`.
4. **No upper bound on** `quantity` — combine with a stock check; also consider a hard cap (e.g., 99) to prevent abuse.
5. **Inconsistent response shapes** — `get_cart` returns `{"data": ...}`, others return `{"message": ...}`. Standardize with `{"success": True, ...}`.
6. **No try/except on** `get_cart`, `update_cart_item`, `remove_cart_item`, `clear_cart` — errors bubble up as generic 500s. Consider wrapping for consistent error responses.
7. `stock` **parsing swallows errors** — `except (TypeError, ValueError): pass` means "if stock data is broken, allow anything." Prefer raising a 500 so the failure is visible.
8. `add_to_cart` **returns no data** — you might want to return the updated row so the frontend can update its UI without a second request.
9. **No cart expiry** — items sit forever. Some stores prune carts older than N days.
10. **Guest carts aren't supported** — some stores let anonymous users build a cart (stored by session) and merge it on login. This API requires auth.
11. `add_to_cart` **doesn't check the current product is still active** — a product may have been deactivated. Consider joining `products` and rejecting inactive ones.
12. **Route order warning** — `@router.delete("/{item_id}")` vs `@router.delete("/")`. Since they're distinct paths, no collision. But if you ever add `@router.delete("/all")`, remember it must come **before** `/{item_id}`.

Would you like me to:

- Show a **hardened** `add_to_cart` with a DB-level atomic upsert and stock enforcement, or
- Draw the **end-to-end cart → checkout → order flow** showing how `cart.py` feeds into `orders.py`?

auth.py
PY 2.91KB

Full Line-by-Line Explanation of `auth.py`

This file defines the **Authentication API** — the front door for users. It handles signup, login, session refresh, logout, current-user identity, and password reset. It's built on top of **Supabase Auth**, which manages users, password hashing, JWT issuance, and email sending for you.

Let's walk through every line.

1. Imports

```python
from fastapi import APIRouter, Depends, HTTPException
from fastapi.security import HTTPAuthorizationCredentials
from pydantic import BaseModel, EmailStr

from app.database import supabase
from app.dependencies import get_current_user, security
```

Import	Purpose
`APIRouter`	Groups the auth routes.
`Depends`	Runs auth dependencies.
`HTTPException`	Returns 4xx errors (e.g., bad credentials → 400).
`HTTPAuthorizationCredentials`	A **typed container** that FastAPI's `HTTPBearer` security scheme fills with the raw token + scheme.
`BaseModel`	Pydantic base for request schemas.

`EmailStr`	Validates email format automatically.
`supabase`	The database + auth client.
`get_current_user`	Custom dependency that verifies the JWT and returns the user.
`security`	A FastAPI `HTTPBearer()` instance (imported from `app.dependencies`). Used to extract the raw token in `logout`.

> **Note:** `security` is a `HTTPBearer` object. You typically define it like `security = HTTPBearer()` in `app/dependencies.py`, then reuse it across routers.

2. Create the Router

```python
router = APIRouter(prefix="/auth", tags=["Authentication"])
```

- `prefix="/auth"` → all routes begin with `/auth`.
- `tags=["Authentication"]` → grouped under "Authentication" in the docs.

3. Request Schemas

```python
class SignUpRequest(BaseModel):  # expected JSON body for signup
    email: EmailStr
    password: str
```

- Body for signup.
- `EmailStr` → Pydantic validates the email format.
- No password-strength rule — **Supabase will enforce its own policy** (or none, if your

project doesn't have one).

- The inline comment clarifies what the class represents.

```python
class LoginRequest(BaseModel):  # expected JSON body for login
    email: EmailStr
    password: str
```

- Same shape as signup — because both take email + password.

```python
class RefreshRequest(BaseModel):
    refresh_token: str
```

- Body for refreshing a session. The client sends the refresh token obtained from login.

```python
class ForgotPasswordRequest(BaseModel):
    email: EmailStr
```

- Body for the password-reset request.

4. POST `/auth/signup` — Register

```python
@router.post("/signup")
def sign_up(payload: SignUpRequest):
    try:
        response = supabase.auth.sign_up({
            "email": payload.email,
            "password": payload.password,
        })
        return {"success": True, "data": response}
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e))
```

- `@router.post("/signup")` → handles `POST /auth/signup`.
- **No auth required** — obviously.
- Calls `supabase.auth.sign_up(...)`, which:
  - Creates a user record.
  - **Hashes the password** using bcrypt (Supabase handles this).
  - Optionally sends a confirmation email (depends on your project settings).
- Returns the raw Supabase response.
- On error (e.g., email already registered, weak password), returns **400**.

> **Common gotcha:** If email confirmation is **enabled**, `sign_up` may not return a session. The user must click a link in their email before they can log in.

5. POST `/auth/login` — Authenticate

```python
@router.post("/login")
def log_in(payload: LoginRequest):
    try:
        response = supabase.auth.sign_in_with_password({
            "email": payload.email,
            "password": payload.password,
        })
        # session holds the access_token (JWT) + refresh_token the client stores
        return {"success": True, "session": response.session, "user": response.user}
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e))
```

- Authenticate with email + password.
- On success, `response.session` contains:
  - `access_token` — a JWT the client sends in `Authorization: Bearer <token>` headers.
  - `refresh_token` — used to get a new access token when the old one expires.
  - `expires_in` — seconds until the access token expires.

- `expires_in` — seconds until the access token expires.

- `response.user` has `id`, `email`, and any metadata.
- The client should store both tokens (securely — ideally in HTTP-only cookies for web).
- On error (bad credentials, unconfirmed email), **400**.

> **Security nuance:** This endpoint is vulnerable to **brute-force/credential stuffing**. In production you'd want rate limiting (e.g., per IP or per email), CAPTCHA, and possibly account lockouts.

6. GET `/auth/me` — Current User Identity

```python
@router.get("/me")
def current_user_info(current_user=Depends(get_current_user)):
    """Return the authenticated user's basic identity."""
    return {
        "success": True,
        "user": {"id": current_user.id, "email": current_user.email},
    }
```

- Auth required via `get_current_user`.
- Returns a **trimmed** view of the user — just `id` and `email`. This is deliberate: don't leak internal Supabase fields (metadata, timestamps, app metadata) to the frontend.
- Useful for:
  - Checking whether a token is still valid.
  - Populating a header avatar menu.

> **Comparison:** `profiles.py` has a similar `/profiles/me`, but that fetches *profile* data. This one fetches *auth identity*. They can diverge (a user can exist without a profile row).

7. POST `/auth/refresh` — Get a New Access Token

```python
@router.post("/refresh")
def refresh_session(payload: RefreshRequest):
    """Exchange a refresh token for a brand new session (new access token)."""
    try:
        response = supabase.auth.refresh_session(payload.refresh_token)
        return {"success": True, "session": response.session, "user": response.user}
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e))
```

- **Purpose:** Access tokens (JWTs) are short-lived (usually 1 hour). When they expire, the client can't use them. Instead of forcing the user to log in again, the client sends its **refresh token** here.
- Supabase returns a **new session** with a fresh access token (and often a rotated refresh token).
- Returns the same shape as `login` — so the client can handle both with the same code.
- On error (expired/revoked/invalid refresh token) → 400.

> **Best practice:** The frontend should refresh **proactively** — e.g., 30 seconds before expiry — not reactively after a failed request. Simpler clients refresh on 401. Either works.

8. POST `/auth/logout` — Revoke Session

```python
@router.post("/logout")
def logout(
    credentials: HTTPAuthorizationCredentials = Depends(security),
    current_user=Depends(get_current_user),
):
    """Revoke the current token server-side (best effort)."""
    try:
        supabase.auth.admin.sign_out(credentials.credentials)
    except Exception:
```

```
    except Exception:
        pass  # JWTs are stateless; the client discarding the token is enough
    return {"success": True, "message": "Logged out"}
```

- **Two dependencies**:
  - `security` → FastAPI's HTTPBearer extracts the raw token from the `Authorization: Bearer ...` header into `credentials`.
  - `get_current_user` → verifies the token and populates `current_user`.
- `credentials.credentials` = the raw JWT string.
- `supabase.auth.admin.sign_out(...)` tries to **revoke** the token server-side.
  - Requires an **admin/service-role** key (not the public anon key). If your app only has anon keys, this would fail.
  - Even then, JWTs are **stateless** — a revoked JWT is only invalidated if Supabase checks a revocation list, which it does for logout.
- The `try/except: pass` swallows errors deliberately:
  - Even if revocation fails, the client should **discard the token anyway**. From the client's point of view, logout "works."
  - This is why the code comment says "best effort".
- Always returns success.

> **Real-world gotcha:** For a stateless JWT, "logging out" is really "client throws away the token." Server-side revocation only matters if:
>
> - A stolen token needs to be invalidated before it expires, or
> - You have a refresh-token rotation strategy.
>
> Many apps skip the server call entirely and just delete the token client-side.

9. POST `/auth/forgot-password` — Send Reset Email

python

```
@router.post("/forgot-password")
def forgot_password(payload: ForgotPasswordRequest):
```

```
    """Send a password-reset email.

    We always return success so an attacker cannot probe which emails exist.
    """
    try:
        supabase.auth.reset_password_email(payload.email)
    except Exception:
        pass
    return {
        "success": True,
        "message": "If that email is registered, a reset link has been sent.",
    }
```

- **The docstring is the key insight**: always return success — whether or not the email exists in the system.
- **Why?** Otherwise, an attacker could send thousands of password-reset requests and, by observing which ones get 200 vs 404, build a list of registered emails. That's called **user enumeration**.
- Supabase's `reset_password_email` sends the actual email — but only if the address is registered.
- Errors are swallowed silently.
- The message is intentionally vague: **"If that email is registered..."** — the classic careful wording.

> **Design principle:** Security-relevant endpoints should not leak whether a resource exists. The same pattern applies to signup ("email already registered" is often softened).

10. Big Picture Summary

Route	Method	Who	Purpose
`/auth/signup`	POST	Anyone	Create a user
`/auth/login`	POST	Anyone	Get access + refresh tokens

`/auth/me`	GET	Logged-in	Get my id/email
`/auth/refresh`	POST	Anyone w/ refresh token	Exchange for new access token
`/auth/logout`	POST	Logged-in	Best-effort revoke
`/auth/forgot-password`	POST	Anyone	Send reset email (no enumeration)

The authentication lifecycle:

```
text

1. POST /auth/signup      → user created
2. POST /auth/login       → get session (access + refresh tokens)
3. GET  /auth/me          → client validates it still works
4. (every request)        → Authorization: Bearer <access_token>
5. access token expires   → POST /auth/refresh → new session
6. user clicks "Logout"   → POST /auth/logout + client discards token
```

Key concepts used:

1. **Delegating auth to Supabase** — password hashing, JWTs, and email sending are all handled for you.
2. **Access token vs refresh token** — short-lived JWT + long-lived refresh token.
3. **Best-effort logout** — tolerate failures because the client will discard the token anyway.
4. **User enumeration prevention** — always say "if that email is registered..."
5. **Trimmed response in** `/me` — don't leak internal fields.
6. **Pydantic** `EmailStr` — validates email format automatically.

11. Bugs / Improvements to Consider

1. **No password-strength enforcement** — Supabase will accept `"123"` unless you

configure it. Add a Pydantic validator (min length, complexity).

2. **No rate limiting** — `/login`, `/signup`, and `/forgot-password` are prime abuse targets. Add IP/email throttling (e.g., `slowapi` or a custom middleware).
3. **No CAPTCHA** on signup/login/forgot — same reason.
4. **Generic 400 for wrong credentials** — an attacker gets the same error for a valid vs. invalid email (which is good) but you might want to differentiate "email not confirmed" to help users. Consider a 401 for bad credentials, 403 for unconfirmed.
5. **Session tokens returned to the client as JSON** — for a **web** frontend, HTTP-only cookies are much safer (they protect against XSS token theft). JSON is fine for mobile apps.
6. `logout` **uses** `supabase.auth.admin.sign_out` — requires a service-role key. If your `supabase` client uses only the anon key, this silently fails, and the "best effort" comment hides that. Consider using the **user-scoped** client (`sign_out()` after setting the session) instead.
7. **No CSRF protection if using cookies** — if you switch to cookies, you'll need CSRF tokens.
8. `/refresh` **doesn't validate token ownership** — irrelevant here because the refresh token *is* the proof, but worth noting: you should reject re-used refresh tokens (Supabase rotates them).
9. **No** `password_confirmation` — nice for user experience but optional.
10. **No email verification enforcement** — if email confirmation is enabled, a user can technically "log in" without confirming. You might want to check `response.user.email_confirmed_at` and reject if missing.
11. **Response shapes inconsistent** — some return `{"session": ..., "user": ...}`, others `{"data": ...}`. Standardize.
12. **No access-token TTL exposure** — clients often want to know when to refresh. Exposing `session.expires_in` and `session.expires_at` directly helps them. (Supabase does include them in `session`, so this is implicitly handled.)
13. `str(e)` **leaks Supabase error details** — an attacker could learn about your project configuration. Sanitize.

Would you like me to:

- Show a **hardened** `auth.py` (rate limiting + password policy + cookie-based sessions), or
- Draw the **end-to-end auth flow** showing how tokens produced here flow into `get_current_user` and the other routers?

This response is AI-generated, for reference only.



Full Line-by-Line Explanation of `analytics.py`

This file defines a **single admin-only endpoint** that returns a small "dashboard summary" — the kind of numbers a store owner wants to see at a glance when they open their admin panel:

1. **Today's revenue** (from paid orders)
2. **How many orders are waiting for PIN entry** (pending M-Pesa payments)
3. **Which products are running low on stock**

Let's walk through it.

1. Imports

```python
from fastapi import APIRouter, HTTPException, Depends
from app.database import supabase
from app.dependencies import require_admin
from datetime import datetime, timedelta
```

Import	Purpose
`APIRouter`	Groups the analytics routes.

`HTTPException`	Returns a 500 on failure.
`Depends`	Runs the admin check before the route.
`supabase`	The database client.
`require_admin`	Auth dependency that only allows admins.
`datetime`	Gets today's date for the revenue filter.
`timedelta`	Date arithmetic — **imported but not used** (see improvements).

> ⚠️ **Unused import:** `timedelta` is imported but never referenced. Harmless, but linters will flag it.

2. Create the Router

```python
router = APIRouter(prefix="/analytics", tags=["analytics"])
```

- `prefix="/analytics"` → all routes here begin with `/analytics`.
- `tags=["analytics"]` → grouped in docs. Note the lowercase `"analytics"` (other files use capitalized tags like `"Orders"`); purely cosmetic.

3. GET `/analytics/summary` — Admin Dashboard Numbers

```python
@router.get("/summary")
def get_analytics_summary(_admin_user = Depends(require_admin)):
    """
    Admin endpoint to get:
    1. Total daily revenue (for today)
```

```
    2. Count of pending orders
    3. Products with low stock (< 5)
    """
```

- `@router.get("/summary")` → handles `GET /analytics/summary`.
- `_admin_user = Depends(require_admin)` → **admin-only**. The underscore prefix means "we need the dependency to run, but don't use its value."
- The docstring lists the three things this endpoint returns.

Body

python

```
    try:
        # 1. Total Daily Revenue
        today = datetime.now().strftime("%Y-%m-%d")
```

- Get **today's date** as a string like `"2026-10-12"`.
- Used later to filter orders created "today" by string prefix matching.

> **Time zone caveat:** `datetime.now()` uses the **server's local time**. If your server runs in UTC but your store operates in Nairobi (UTC+3), "today" will be wrong around midnight. Consider using `datetime.now(timezone.utc)` or a fixed timezone like `ZoneInfo("Africa/Nairobi")`.

python

```
        # Assuming created_at is stored in ISO format
        orders_response = supabase.table("orders").select("total_amount, status, create
d_at").execute()
        orders = orders_response.data or []
```

- Fetch **all** orders, but only the three columns we need.
- `orders_response.data or []` → if `data` is `None` (empty), use `[]` instead. A safe fallback.

> ⚠️ **Performance concern:** This pulls **every order ever created** just to compute today's revenue. As the store grows, this becomes slow and bandwidth-heavy. Better

> revenue. As the store grows, this becomes slow and bandwidth-heavy. Better approach: push the filtering into the DB query with `.eq("status", "Paid").gte("created_at", today)`.

python

```
        daily_revenue = 0.0
        for order in orders:
            if not isinstance(order, dict):
                continue

            status = order.get("status")
            created_at = order.get("created_at")
            total_amount = order.get("total_amount")
```

- Start the accumulator at `0.0`.
- Loop over each order.
- **Defensive check** — skip anything that isn't a dict.
- Extract the three fields via `.get(...)` (safe — returns `None` if missing).

python

```
            if status == "Paid" and isinstance(created_at, str) and created_at.startswi
th(today):
                try:
                    if isinstance(total_amount, (int, float, str)) and not isinstance(t
otal_amount, bool):
                        daily_revenue += float(total_amount)
                except (TypeError, ValueError):
                    continue
```

This is the heart of the revenue calculation. Filter conditions:

1. `status == "Paid"` → only count orders that were actually paid. `"Pending PIN"`, `"Cancelled"`, etc. are excluded.
2. `isinstance(created_at, str)` → only if `created_at` is a string (avoids errors on `None`).
3. `created_at.startswith(today)` → an ISO timestamp like `"2026-10-12T14:35:00"` starts with `"2026-10-12"`, so it matches today. **This is why the code comment says "assuming created_at is stored in ISO format."**

4. `isinstance(total_amount, (int, float, str)) and not isinstance(total_amount, bool)` → allow only numeric-looking values, and **explicitly reject booleans** (since `True == 1` in Python — defensive again).
5. `float(total_amount)` → convert string or number to a float for addition.
6. `except (TypeError, ValueError): continue` → if conversion fails (e.g., `"abc"`), skip this order.

Result: `daily_revenue` accumulates today's paid totals.

2. Pending Orders

```python
pending_orders = [
    order for order in orders
    if isinstance(order, dict) and order.get("status") == "Pending PIN"
]
```

- A **list comprehension** filtering `orders`.
- Keeps orders where:
  - It's a dict.
  - Status is exactly `"Pending PIN"`.
- **No date filter here** — it counts pending orders regardless of when they were created. Arguably this is what you'd want (an order pending for 3 days is still pending), but worth being aware of.
- The count is only the **length** of this list.

> **Inefficiency:** We're looping over all orders **twice** (once for revenue, once for pending). A single loop could compute both. Minor, but noticeable at scale.

3. Low Stock Products

```python
products_response = supabase.table("products").select("name, stock_quantity").l
```

```
t("stock_quantity", 5).execute()
        low_stock_products = products_response.data or []
```

- `.lt("stock_quantity", 5)` → "less than 5". This **is** done in the DB, which is good.
- Selects only `name` and `stock_quantity`.
- Falls back to `[]` if empty.

> **Bug risk:** If `stock_quantity` is stored as text (e.g., `"3"`), `.lt(...)` comparisons may behave unexpectedly — lexicographic vs. numeric. Make sure the column is numeric in Postgres.

4. The Response

python

```
        return {
            "daily_revenue": daily_revenue,
            "pending_orders_count": len(pending_orders),
            "low_stock_products": low_stock_products
        }
```

- A compact JSON object with the three numbers/lists.
- Note: no `"success": True` wrapper here — inconsistent with most other files. Frontends must adapt.

5. Error Handling

python

```
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

- Any unexpected error → 500 with the message.
- Standard and correct pattern.

4. Big Picture Summary

Route	Method	Who	Purpose
`/analytics/summary`	GET	Admin	Daily revenue + pending count + low-stock list

Where this fits:

- The **admin dashboard** calls this once on load to render headline cards:
  - "Today's Revenue: KES 45,200"
  - "Pending Orders: 12"
  - "Low Stock: 4 products"
- Likely combined with `/orders/all` (list) to show recent activity.

Key concepts used:

1. **Admin-only dependency** — `require_admin` gates the endpoint.
2. **Application-side aggregation** — daily revenue is computed in Python, not SQL.
3. **String prefix matching for dates** — relies on ISO 8601 timestamps.
4. **Defensive type checks** — `isinstance`, `.get()`, boolean exclusion.
5. **DB-side filter for stock** — `.lt("stock_quantity", 5)` runs in Postgres.
6. **List comprehension for counting** — clean and readable.

5. Bugs / Improvements to Consider

1. **Pulls all orders into memory** — this will slow down as orders grow. Better: use DB aggregation. Supabase supports `.gte("created_at", today)` and a Postgres **RPC** for `SUM(total_amount)`:

```sql
create or replace function daily_revenue(p_date date)
returns numeric as $$
```

```
  select coalesce(sum(total_amount), 0)
  from orders
  where status = 'Paid' and created_at::date = p_date;
$$ language sql;
```

Then `supabase.rpc("daily_revenue", {"p_date": today})`.

2. **Time zone bug** — `datetime.now()` is server-local. Use `datetime.now(timezone.utc)` or the store's timezone.
3. **String prefix date matching is fragile** — if `created_at` is ever stored with a different format (or a different timezone offset), the comparison silently fails. Parsing to a `date` object is more robust.
4. **Looping twice over** `orders` — merge into a single loop for efficiency.
5. `timedelta` **is imported but unused** — remove.
6. **No** `"success": True` **wrapper** — inconsistent with most of the codebase. Frontends get confused.
7. `.lt("stock_quantity", 5)` **has the** `5` **hardcoded** — you may want a config or a query parameter `?threshold=10`.
8. **No product image or id in low stock list** — the admin probably wants `id`, `image_url`, and stock count to act on it.
9. **Response has no timestamp** — the frontend can't tell when the data was computed. Add `"generated_at": datetime.now().isoformat()`.
10. **No pagination on low stock** — if 500 products are low, they all come back. Add `.order("stock_quantity").limit(20)` or similar.
11. `str(e)` **in error response** — leaks internals. Log server-side, return generic.
12. **No caching** — a dashboard summary computed on every page load can be cached for 30–60 seconds. Consider `Cache-Control` or an in-process TTL.
13. `"Pending PIN"` **is hardcoded string** — if you ever rename the status, this breaks. Consider a central `Status` enum module.
14. **No check that** `daily_revenue` **reflects completed-then-refunded orders** — a refunded order would still count. You'd need a refunds model to handle this correctly.

2. A Quick Walkthrough of the Supabase

6. A Quick "Improved Version" of the Core Loop

If you want the same output but with a **single pass** and cleaner separation:

```python
@router.get("/summary")
def get_analytics_summary(_admin_user = Depends(require_admin)):
    try:
        today = datetime.now(timezone.utc).strftime("%Y-%m-%d")

        orders = (supabase.table("orders")
                  .select("total_amount, status, created_at")
                  .gte("created_at", today)
                  .execute().data or [])

        daily_revenue = 0.0
        pending_count = 0
        for order in orders:
            if not isinstance(order, dict):
                continue
            status = order.get("status")
            if status == "Pending PIN":
                pending_count += 1
            elif status == "Paid":
                amt = order.get("total_amount")
                if isinstance(amt, (int, float)) and not isinstance(amt, bool):
                    daily_revenue += float(amt)

        low_stock = (supabase.table("products")
                     .select("id, name, stock_quantity, image_url")
                     .lt("stock_quantity", 5)
                     .order("stock_quantity")
                     .execute().data or [])

        return {
            "success": True,
            "generated_at": datetime.now(timezone.utc).isoformat(),
            "data": {
                "daily_revenue": round(daily_revenue, 2),
                "pending_orders_count": pending_count,
                "low_stock_products": low_stock,
            }
        }
```

```
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

Changes: single pass, DB-side date filter, timezone-aware, consistent response shape, better low-stock output, rounded revenue.

Would you like me to:

- Write the **Postgres RPC** for daily revenue (and how to call it from Python), or
- Do a **final cross-file summary** of the whole codebase with a "cheat sheet" of patterns and conventions used across all 12 files?



Full Line-by-Line Explanation of `ai.py`

This file defines an **AI chat assistant** endpoint. It lets a logged-in user chat with a large language model (LLM) — a Llama 3.1 model, via **OpenRouter** — that plays the role of a **friendly phone-sales assistant**. The AI is given the current product inventory as context so it can recommend real products.

It does four big things:

1. Stores the user's message.
2. Fetches the store's inventory.
3. Builds a system prompt + chat history, sends it to the LLM.
4. Stores the AI's reply and returns it.

Let's walk through every line.

1. Imports

```python
```

python

```python
import os
import httpx
from fastapi import APIRouter as FastAPIRouter, HTTPException, Depends
from pydantic import BaseModel
from typing import List, Dict, Optional
from app.database import supabase
from app.dependencies import get_current_user
from dotenv import load_dotenv
```

Import	Purpose
`os`	Read environment variables.
`httpx`	Async HTTP client — makes the outbound call to OpenRouter.
`APIRouter as FastAPIRouter`	Aliased — see below.
`HTTPException , Depends`	Standard FastAPI.
`BaseModel`	Pydantic schema.
`List , Dict , Optional`	Type hints.
`supabase`	DB client.
`get_current_user`	Auth dependency.
`load_dotenv`	Loads `.env` variables.

> **Why** `APIRouter as FastAPIRouter` **?** Purely to avoid a name clash with something else in the module (perhaps a `router` variable, or a project convention). It's cosmetic — `FastAPIRouter` behaves exactly like `APIRouter` . This alias pattern often appears when people copy code between projects.

> ⚠️ **Unused imports:** `Optional` and `Dict` are imported but never used — only `List` is. Linters would flag these.

2. Load Environment + Create Router

```python
load_dotenv()
router = FastAPIRouter(prefix="/ai", tags=["AI Assistant"])
```

- `load_dotenv()` → reads `.env` and sets OS env vars (so `os.environ.get(...)` works below).
- Router with prefix `/ai` and tag `"AI Assistant"`.

> **Best practice note:** `load_dotenv()` is often called once in `main.py`. Calling it per-file is harmless but slightly redundant.

3. API Key

```python
OPENROUTER_API_KEY = os.environ.get("OPENROUTER_API_KEY")
```

- Reads the OpenRouter API key from env vars **at import time**.
- If missing, this is `None` — and every API call later will fail with a 401.
- **Better**: fail fast at startup if missing, rather than letting it silently break at runtime.

4. Request Schema

```python
class ChatRequest(BaseModel):
    messages: List[Dict[str, str]]
```

- The expected request body: a **list of chat messages**, each a dict with (at least) `role` and `content`.

- Example:

```
json

{
  "messages": [
    {"role": "user", "content": "I want a phone under $500"}
  ]
}
```

- Typically the frontend sends the **entire conversation** so far, but the backend only uses the last message + its own DB history.

5. POST `/ai/chat` — Chat With the Assistant

```
python

@router.post("/chat")
async def chat_with_ai(payload: ChatRequest, current_user = Depends(get_current_user)):
    if not payload.messages:
        raise HTTPException(status_code=400, detail="Messages cannot be empty")
```

- `@router.post("/chat")` → handles `POST /ai/chat`.
- Auth required.
- Reject empty message lists.

Step 1 — Store the User's Message

```
python

    # 1. Get the latest user message and store it
    user_message = payload.messages[-1]
    supabase.table("chat_history").insert({
        "user_id": current_user.id,
        "role": "user",
        "content": user_message["content"]
    }).execute()
```

- `payload.messages[-1]` → the **last** item (the newest user message).
- Insert it into a `chat_history` table with:
  - `user_id` → from the token (never trusted from the body).
  - `role` → `"user"`.
  - `content` → the message text.
- This persists the conversation so we can retrieve it later for context.

> **Risk:** `user_message["content"]` uses direct indexing — if the client sends `{"role": "user"}` with no `content`, this raises `KeyError` and returns a 500. Better: `user_message.get("content", "")` and validate it.

Step 2 — Fetch Product Inventory for Context

python

```
    # 2. Fetch active products for context
    products_res = supabase.table("products").select("name, price, brand, descriptio
n").execute()
    product_rows = products_res.data or []
    inventory_lines = []
```

- Fetch all products (name, price, brand, description).
- `product_rows or []` → safe fallback.
- Prepare an empty list to build the human-readable inventory.

> ⚠️ **Missing filter:** This fetches *all* products, including **inactive** ones (if such a column exists). Consider adding `.eq("active", True)` — otherwise the AI may recommend out-of-catalog items.

> ⚠️ **Scale concern:** If the store has thousands of products, this builds a giant prompt and eats tokens. For a demo it's fine; for production you'd want a search/RAG step.

python

```
    for product in product_rows:
        if not isinstance(product, dict): continue
```

```
            if not isinstance(product, dict): continue
            name = product.get("name") or "Unknown"
            brand = product.get("brand") or "Unknown"
            price = product.get("price")
            if not isinstance(price, (int, float)): price = "0.00"
            description = product.get("description") or "No description"
            inventory_lines.append(f"- {name} ({brand}): ${price} - {description}")
    inventory_str = "\n".join(inventory_lines)
```

- Loop through each product defensively.
- **Falsy fallbacks** — `or "Unknown"` means "if empty or None."
- Price: if it isn't numeric, default to the string `"0.00"` so formatting doesn't crash.
- Build lines like:

```
text

- iPhone 15 (Apple): $999.99 - The latest iPhone with a better camera
```

- Join all lines into one multi-line string.

> ⚠️ **Missing `id`:** The system prompt below asks the AI to return a `product_id`, but this inventory doesn't include product IDs. So the AI **cannot** actually include real IDs — it will hallucinate. This is a bug worth fixing (add `id` to the select and to each line).

Step 3 — Build the System Prompt

```
python

    system_prompt = f"""You are an expert, friendly salesperson assistant for X-UK, a p
hone retailer.
    Your goal is to guide the user to the best phone recommendation based on their need
s.

    Inventory:
    {inventory_str}

    Conversation Flow:
    1. If vague, ask brief qualifying questions (budget, use, brand) one at a time.
    2. When suggesting phones, you MUST provide the product_id so the UI can add it to
the cart.
```

```
    3. Format your recommendations as a JSON object array for products:
       [
         {{"name": "iPhone 17", "product_id": "UUID", "reason": "Reason here"}},
         ...
       ]
    4. Keep text concise and use bullet points.
    5. Always end by asking if they need differences explained between iPhone and other
s."""
```

- **f-string** with `inventory_str` injected in.
- The **system prompt** tells the model:
  - Its **role** ("friendly sales assistant for X-UK").
  - The **store context** (inventory list).
  - **How to behave** (ask qualifying questions, be concise).
  - **The output format** (a JSON array with `name`, `product_id`, `reason`).
  - **How to end** the conversation.
- `{{ ... }}` escapes curly braces so they appear literally in the f-string rather than being interpreted.

> **Design pattern:** Structured output via natural language instructions. This is a "poor man's" function calling — you're asking the model to emit JSON. It works but is brittle; the model might emit prose or malformed JSON. Modern APIs support `response_format: {"type": "json_object"}` or **tool calling** for reliability.

Step 4 — Retrieve Chat History

python

```python
    # 3. Retrieve last 6 messages
    history_res = supabase.table("chat_history") \
        .select("role, content") \
        .eq("user_id", current_user.id) \
        .order("created_at", desc=True) \
        .limit(6) \
        .execute()
    # Reverse to get oldest first for AI context
    history = history_res.data[::-1]
```

- Fetch the **last 6 messages** for this user (newest first, due to `desc=True`).
- `[::-1]` reverses the list so it's **oldest first** — the order the LLM expects.
- Each element is `{"role": "user"|"assistant", "content": "..."}`.

> **Why 6?** A balance between context and token cost. The last 6 messages (≈ 3 exchanges) is often enough for short conversations, and cheap to send.

> ⚠️ **Race condition subtlety:** We already inserted the just-now user message in Step 1. So the last 6 includes that message — good, because the LLM needs to see what the user just asked.

> ⚠️ **Bug risk if `created_at` ties:** If two messages are inserted in the same second (user message + assistant reply), their ordering may be ambiguous. Consider a `sequence` integer column, or millisecond timestamps.

Step 5 — Assemble the Messages

python

```
    # 4. Construct messages for AI
    messages = [{"role": "system", "content": system_prompt}] + history
```

- Prepend the system prompt, then append the history.
- Final shape (OpenAI-compatible):

json

```
[
  {"role": "system",    "content": "<system prompt>"},
  {"role": "user",      "content": "I want a phone under $500"},
  {"role": "assistant", "content": "..."},
  {"role": "user",      "content": "..."}
]
```

> **Note:** `payload.messages` is *not* used here — the frontend's sent history is ignored in favor of DB history. Only the last item (Step 1) is used. This is a design choice: trust the server's record over the client's. It's safer, but the client may have context that the DB

> server's record over the client's. It's safer, but the client may have context that the DB doesn't (e.g., if some messages weren't saved).

Step 6 — Call the LLM

```python
    # 5. Call AI
    async with httpx.AsyncClient() as client:
        response = await client.post(
            "https://openrouter.ai/api/v1/chat/completions",
            headers={"Authorization": f"Bearer {OPENROUTER_API_KEY}"},
            json={"model": "meta-llama/llama-3.1-8b-instruct", "messages": messages},
            timeout=30.0
        )
```

- OpenRouter exposes an **OpenAI-compatible** `/chat/completions` endpoint, so this looks like a call to OpenAI.
- Bearer auth with the API key.
- `model` → `meta-llama/llama-3.1-8b-instruct` (an 8-billion-parameter Llama 3.1).
  - **8B is small** — it's fast and cheap, but not great at following complex JSON-emission instructions. 70B or a bigger model would follow format better.
- `timeout=30.0` → kill the call after 30 seconds.

> ⚠️ **Optional headers ignored:** OpenRouter recommends setting `HTTP-Referer` and `X-Title` headers for dashboard attribution and rate-limit tiers. Not required, but useful.

```python
    if response.status_code != 200:
        error_detail = response.text
        raise HTTPException(status_code=500, detail=f"AI Service Error: {error_detai
l}")
```

- If the LLM service returned anything but 200, return 500 with the raw text.
- **Security note:** `response.text` may include API details. In production, log it server-side and return a generic message.

and return a generic message.

Step 7 — Parse and Store the AI's Reply

```python
    # 6. Store AI response
    ai_data = response.json()
    ai_content = ai_data["choices"][0]["message"]["content"]
    supabase.table("chat_history").insert({
        "user_id": current_user.id,
        "role": "assistant",
        "content": ai_content
    }).execute()

    return ai_data
```

- Parse the JSON response.
- **Navigate the OpenAI-compatible shape**:

```text
ai_data["choices"][0]["message"]["content"]
```

  - `choices` → array of candidate completions (usually one).
  - `[0]` → the first.
  - `message.content` → the reply text.
- Insert the assistant reply into `chat_history` (same pattern as Step 1).
- **Return the full raw response** to the client — including `usage` stats, `id`, `model`, etc.

> ⚠️ **Direct indexing is risky:** If OpenRouter returns a different shape on an edge case, `KeyError` / `IndexError` → 500. Defensive `.get(...)` would be safer.

> ⚠️ **Return shape inconsistency:** The frontend gets the raw OpenRouter envelope, not a simplified `{"role": "assistant", "content": "..."}`. Clients have to parse OpenRouter's format. A cleaner API would return something like:

```python
```

```
return {
    "success": True,
    "message": {"role": "assistant", "content": ai_content}
}
```

6. Big Picture Summary

Route	Method	Who	Purpose
`/ai/chat`	POST	Logged-in user	Chat with an AI sales assistant

The full flow:

1. User sends `POST /ai/chat` with `{"messages": [..., {"role": "user", "content": "..."}]}`.
2. Backend saves the user message.
3. Backend fetches inventory → builds a system prompt.
4. Backend fetches the last 6 messages from DB → assembles the LLM prompt.
5. Backend calls OpenRouter → gets a completion.
6. Backend saves the AI reply.
7. Backend returns the raw OpenRouter response.

Key concepts used:

1. **LLM prompt engineering** — role + context + format instructions + behavior rules.
2. **Context injection** — the store's inventory is embedded in the system prompt.
3. **Conversation persistence** — messages are saved and replayed on each turn.
4. **Stateless HTTP LLM API** — OpenRouter takes the whole conversation every call.
5. **OpenAI-compatible API** — uses the same request/response schema as OpenAI's API.
6. **Defensive DB reads** — `or []` and `isinstance` checks prevent crashes.
7. **Environment-sourced API key** — never hard-coded.

7. Bugs / Improvements to Consider

1. **Missing product `id` in inventory** — the system prompt asks the AI to return a `product_id`, but the inventory doesn't include IDs. The AI will hallucinate them. **Fix**: add `id` to the select and each inventory line:

   ```
   python

   products_res = supabase.table("products").select("id, name, price, brand, descriptio
   n").execute()
   ...
   inventory_lines.append(f"- {name} (id: {product.get('id')}) ({brand}): ${price} - {de
   scription}")
   ```

2. **No filtering on active products** — inactive products may be recommended.
3. **No size limit on inventory** — thousands of products → giant prompt, huge cost.
4. **Prompt injection risk** — user messages are inserted into the conversation, but at least they're labeled `role: "user"`, so they can't overwrite the system prompt in well-behaved models. Still, a malicious user could try to get the AI to leak the system prompt or lie about prices. Consider adding guardrails.
5. **Hallucination risk** — the AI might invent products, prices, or "reason" text. For a store, this matters. **Consider:** generate the recommendation JSON server-side from a database query after the AI returns a product name/ID, then stitch them together.
6. **Return shape is raw OpenRouter** — clients shouldn't have to parse `choices[0].message.content`. Return a simplified object.
7. **No rate limiting** — each call costs money and consumes tokens. Rate-limit per user (e.g., 20/hour).
8. **No token-usage cap** — track `ai_data["usage"]` per user, and stop serving when a budget is exceeded.
9. `payload.messages` **is barely used** — only the last message is stored; the rest is ignored. Consider either trusting it (safer: no) or validating that the client isn't sending too many.
10. `user_message["content"]` **may** `KeyError` — use `.get(...)` with validation.

11. `ai_data["choices"][0]["message"]["content"]` **may fail** — wrap in `.get(...)` chains.
12. `OPENROUTER_API_KEY` **may be** `None` — fail fast at import/startup.
13. `response.text` **in the 500 detail** — leaks service internals; sanitize.
14. `timeout=30.0` — for a chat UX this is acceptable, but for slow models it's short. Also, `httpx.Timeout` with connect/read/write stages would be more precise.
15. **No streaming** — the response is delivered all at once. A streaming SSE endpoint would feel much more responsive to users.
16. **Chat history grows unbounded** — you have `limit(6)` for what you *send*, but nothing prunes old rows. Add retention (e.g., delete >90 days).
17. **Unused imports** — `Optional`, `Dict` should be removed.
18. **No try/except around the LLM call** — network failures bubble up as 500s. Consider wrapping with a friendlier message.

8. A Quick "Fix List" in Code Form

The most impactful fixes:

```
python

# Include ids in inventory so the AI can reference real products
products_res = (
    supabase.table("products")
    .select("id, name, price, brand, description")
    .eq("active", True)
    .limit(200)          # cap prompt size
    .execute()
)
...
inventory_lines.append(f"- {name} (id: {product.get('id')}) ({brand}): ${price} - {desc
ription}")

# Fail fast if the key is missing
OPENROUTER_API_KEY = os.environ.get("OPENROUTER_API_KEY")
if not OPENROUTER_API_KEY:
    raise RuntimeError("OPENROUTER_API_KEY is not set")

# Defensive parsing of the LLM reply
```

```
# Defensive parsing of the LLM reply
try:
    ai_content = ai_data["choices"][0]["message"]["content"]
except (KeyError, IndexError, TypeError):
    raise HTTPException(status_code=502, detail="AI returned an unexpected response")

# Cleaner response shape
return {
    "success": True,
    "message": {"role": "assistant", "content": ai_content},
    "usage": ai_data.get("usage"),
}
```

Would you like me to:

- Show a **version with structured output** (function calling / JSON mode) so the AI reliably returns real `product_id` s from the DB, or
- Do a **full codebase wrap-up**: a single cheat sheet that summarizes every file (auth, patterns, conventions, cross-file flows like cart → checkout → payment → callback → analytics)?